Undergraduate Introduction to Data Analysis

Teaching Document

Undergraduate Introduction to Data Analysis

Probability, Statistics, Hypothesis Testing, Significance Testing, and Simple Regression with Python

Author

Teaching Document

1 How to use this document

This document introduces undergraduate data analysis from the foundations of probability and **statistics**. It uses Python to make the ideas concrete through simulation, **visualization**, and analysis.

By the end, students should be able to:

1. Explain **random variables**, **probability distributions**, expectation, variance, and sampling variability.
2. Distinguish descriptive **statistics** from **statistical inference**.
3. State null and alternative hypotheses and choose an appropriate test statistic.
4. Interpret p-values, significance levels, **confidence intervals**, Type I error, Type II error, and statistical power.
5. Fit, interpret, and diagnose a **simple linear regression** model.
6. Explain why statistical significance is not the same as practical importance or causal proof.

1.1 Python setup

The examples use common scientific Python packages.

```
# If needed, install these in a terminal before rendering this document:
# python -m pip install numpy pandas matplotlib scipy statsmodels

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm

rng = np.random.default_rng(42)
```

Throughout this document, we set a random seed so that simulated examples are reproducible. In real analysis, reproducibility also means saving the data source, documenting cleaning choices, and recording package versions.

```
import sys
print("Python:", sys.version.split()[0])
```

```
print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
```

```
Python: 3.14.6
NumPy: 2.5.0
pandas: 3.0.3
```

2 The logic of data analysis

A useful way to organize introductory data analysis is:

```
flowchart LR
  A[Real-world question] --> B[Data-generating process]
  B --> C[Observed sample data]
  C --> D[Descriptive statistics and visualization]
  D --> E[Statistical model]
  E --> F[Inference]
  F --> G[Interpretation and decision]
  G --> H[New questions]
```


```
flowchart LR
  A[Real-world question] --> B[Data-generating process]
  B --> C[Observed sample data]
  C --> D[Descriptive statistics and visualization]
  D --> E[Statistical model]
  E --> F[Inference]
  F --> G[Interpretation and decision]
  G --> H[New questions]
```


Listing 1:

The key challenge is that we rarely observe an entire population. Instead, we observe a sample. Probability helps us reason about what samples tend to look like when a model is true. **Statistics** uses observed samples to estimate unknown quantities and assess uncertainty.

3 Probability foundations

3.1 Probability as long-run relative frequency and uncertainty

Probability measures how likely an event is. For an event \(A\),

\[0 \le P(A) \le 1. \]

If two events \(A\) and \(B\) cannot occur together, they are mutually exclusive, and

\[P(A \cup B) = P(A) + P(B). \]

If two events are independent, knowing that one occurred does not change the probability of the other:

\[P(A \cap B) = P(A)P(B). \]

In data analysis, probability often describes a **data-generating process**: the process that could have produced the data we observed.

3.2 Random variables

A **random variable** is a numerical outcome of a random process. Examples:

- $X = 1$ if a coin lands heads and $X = 0$ otherwise.
- Y = height of a randomly selected student.
- T = time until a machine fails.

Random variables can be discrete or continuous.

A **discrete random variable** takes countable values, such as 0, 1, 2, 3. A **continuous** random variable can take values along an interval, such as any positive real number.

3.3 Example: simulating coin flips

For a fair coin, let $X = 1$ for heads and $X = 0$ for tails. Then X follows a Bernoulli distribution with probability $p = 0.5$.

```
n = 20
coin_flips = rng.binomial(n=1, p=0.5, size=n)
coin_flips
```

```
array([1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1])
```

The sample proportion of heads is:

```
coin_flips.mean()
```

```
np.float64(0.6)
```

With only 20 flips, the sample proportion may not be exactly 0.5. Random samples vary.

```
n_flips = 5_000
flips = rng.binomial(n=1, p=0.5, size=n_flips)
running_mean = np.cumsum(flips) / np.arange(1, n_flips + 1)

plt.figure(figsize=(8, 4.5))
plt.plot(running_mean)
plt.axhline(0.5, linestyle="--", linewidth=1)
plt.xlabel("Number of coin flips")
plt.ylabel("Sample proportion of heads")
plt.title("Law of large numbers in action")
plt.show()
```

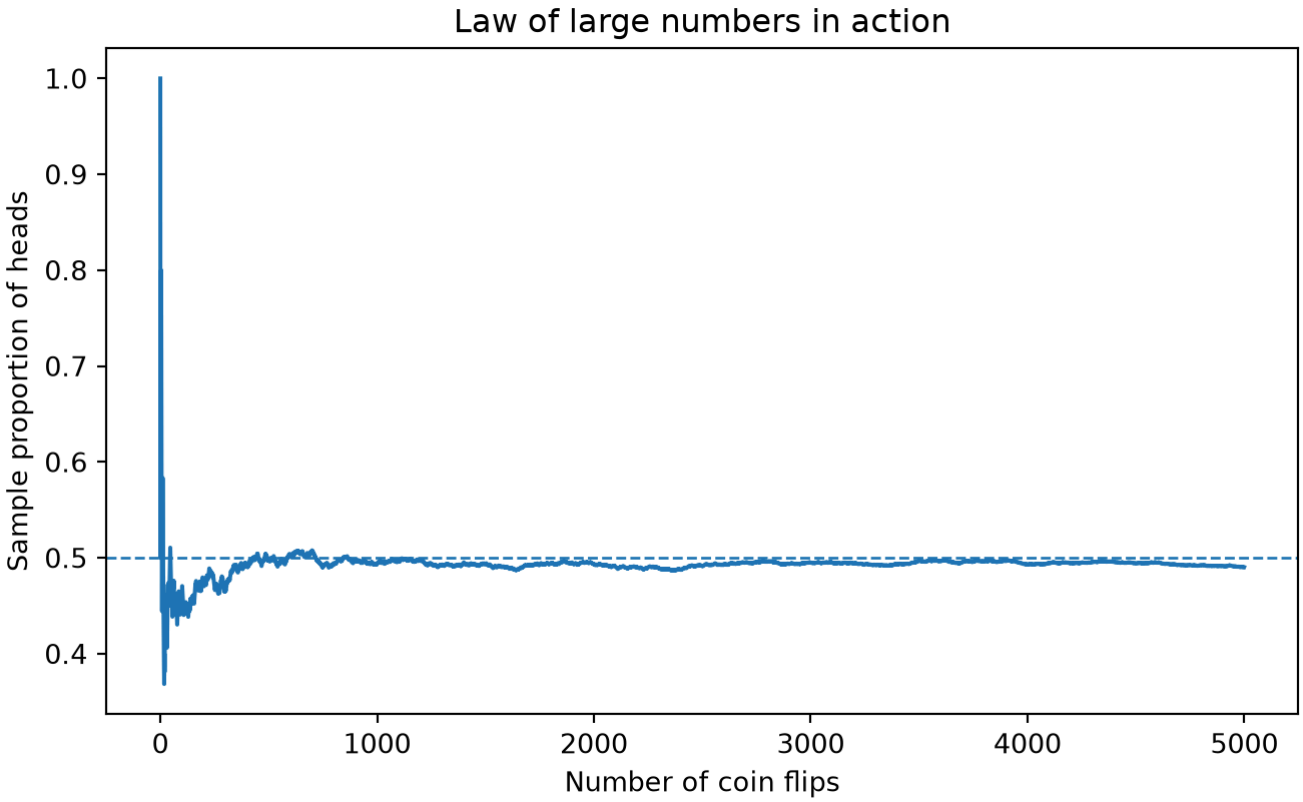


Figure 1: Figure 1: The sample proportion of heads converges toward the true probability as the number of flips increases.

The law of large numbers says that, under suitable conditions, a sample average tends to get closer to the population mean as sample size grows.

3.4 Probability distributions

A **probability distribution** tells us the possible values of a **random variable** and how likely they are.

Common distributions in introductory data analysis include:

Distribution	Type	Typical use	Parameters
Bernoulli	Discrete	One yes/no outcome	\(p\)
Binomial	Discrete	Number of successes in \(n\) independent yes/no trials	\(n, p\)
Normal	Continuous	Approximate model for many measurements and averages	\(\mu, \sigma\)
Student's \(t\)	Continuous	Inference for means when population variance is unknown	degrees of freedom
Chi-square	Continuous	Inference about variances; model comparison	degrees of freedom
F	Continuous	Comparing model fit; ANOVA; regression	degrees of freedom

3.5 Expectation and variance

The expected value of a **random variable** is its probability-weighted average. For a discrete random variable,

\[E[X] = \sum_x xP(X=x). \]

Variance measures spread around the expected value:

\[\operatorname{Var}(X) = E[(X - E[X])^2]. \]

The standard deviation is the square root of the variance:

\[\operatorname{SD}(X) = \sqrt{\operatorname{Var}(X)}. \]

For a Bernoulli **random variable** \(X \sim \operatorname{Bernoulli}(p)\),

\[E[X] = p, \qquad \operatorname{Var}(X)=p(1-p). \]

3.6 Visualizing distributions

```
x = np.linspace(-5, 5, 400)
plt.figure(figsize=(8, 4.5))
for sigma in [0.5, 1, 2]:
    y = stats.norm.pdf(x, loc=0, scale=sigma)
```

```
    plt.plot(x, y, label=f"$\\sigma={sigma}$")
plt.xlabel("x")
plt.ylabel("Density")
plt.title("Normal distributions")
plt.legend()
plt.show()
```

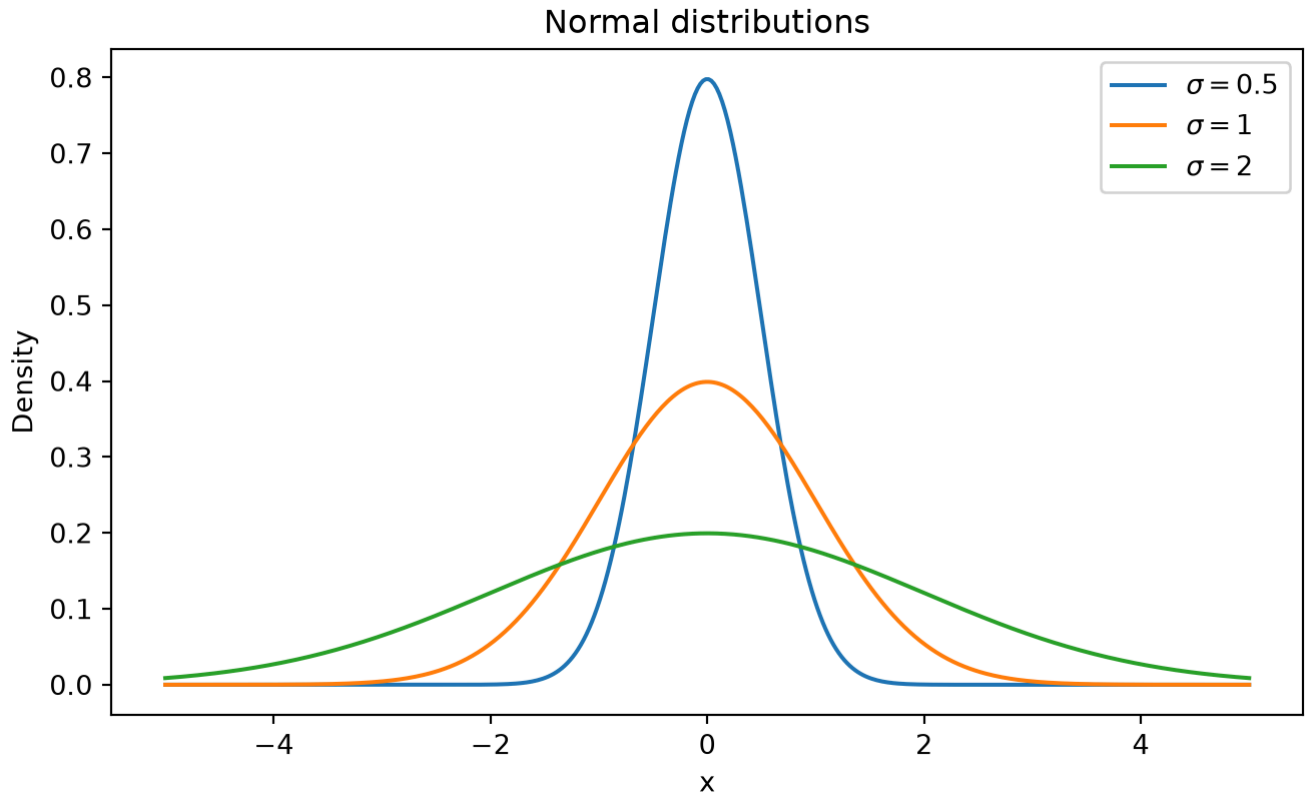


Figure 2: Figure 2: Normal distributions with the same mean but different standard deviations. Larger standard deviation means more spread.

For continuous **variables**, the curve is a density, not a direct probability. Probabilities correspond to areas under the curve.

```
# Probability that a standard normal variable is less than 1.96
stats.norm.cdf(1.96)
```

```
np.float64(0.9750021048517795)
```

For a standard normal variable, about 95% of the distribution lies between −1.96 and 1.96.

```
stats.norm.cdf(1.96) - stats.norm.cdf(-1.96)
```

```
np.float64(0.950004209703559)
```

4 From populations to samples

4.1 Population, sample, parameter, statistic

A **population** is the full group or process we want to understand. A **sample** is the data we observe.

A **parameter** is a numerical feature of the population, such as the population mean \(\mu\). A **statistic** is a numerical feature computed from a sample, such as the sample mean \(\bar{x}\).

Concept	Population	Sample
Mean	\(\mu\)	\(\bar{x}\)
Variance	\(\sigma^2\)	\(s^2\)
Standard deviation	\(\sigma\)	\(s\)
Proportion	\(p\)	\(\hat{p}\)
Regression slope	\(\beta_1\)	\(\hat{\beta}_1\)

Statistical inference asks: what can we say about the population using the sample?

4.2 Sampling variability

Even when the population does not change, different random samples produce different **statistics**.

```
population_mean = 100
population_sd = 15
sample_size = 25
n_samples = 1_000

sample_means = np.array([
    rng.normal(population_mean, population_sd, size=sample_size).mean()
    for _ in range(n_samples)
])

plt.figure(figsize=(8, 4.5))
plt.hist(sample_means, bins=30, edgecolor="black")
plt.axvline(population_mean, linestyle="--", linewidth=1)
plt.xlabel("Sample mean")
plt.ylabel("Frequency")
plt.title("Sampling distribution of the sample mean")
plt.show()
```

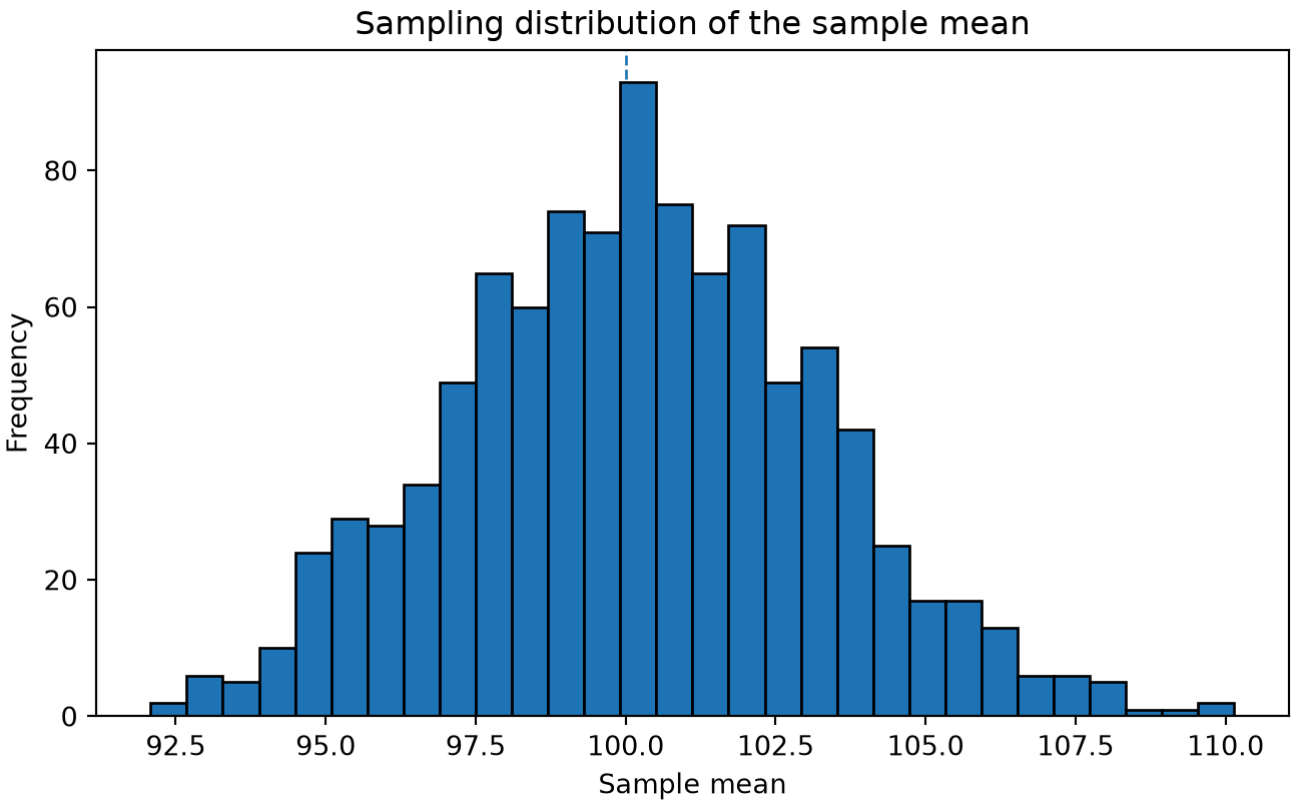


Figure 3: Figure 3: Different samples from the same population produce different sample means.

The distribution of a statistic over repeated samples is called a **sampling distribution**.

4.3 Standard error

The standard deviation of a sampling distribution is called a **standard error**.

For the sample mean,

\[SE(\bar{X}) = \frac{\sigma}{\sqrt{n}}. \]

Because \(\sigma\) is usually unknown, we estimate it with the sample standard deviation \(s\):

\[\widehat{SE}(\bar{X}) = \frac{s}{\sqrt{n}}. \]

```
observed_sample = rng.normal(population_mean, population_sd, size=sample_size)
xbar = observed_sample.mean()
s = observed_sample.std(ddof=1)
se = s / np.sqrt(sample_size)

pd.DataFrame({
    "quantity": ["sample mean", "sample standard deviation", "estimated standard
error"],
    "value": [xbar, s, se]
})
```

	quantity	value
0	sample mean	92.023431
1	sample standard deviation	15.736199
2	estimated standard error	3.147240

4.4 Central limit theorem

The central limit theorem says that, under broad conditions, the sampling distribution of the sample mean becomes approximately normal as sample size increases, even if the original data are not normal.

```
# Exponential data are right-skewed. Their sample means become increasingly
normal as n grows.
for n in [2, 5, 30]:
    means = np.array([rng.exponential(scale=1, size=n).mean() for _ in
range(5_000)])
    plt.figure(figsize=(7, 4))
    plt.hist(means, bins=40, density=True, edgecolor="black")
    plt.xlabel("Sample mean")
    plt.ylabel("Density")
    plt.title(f"Sampling distribution of the mean, n={n}")
    plt.show()
```

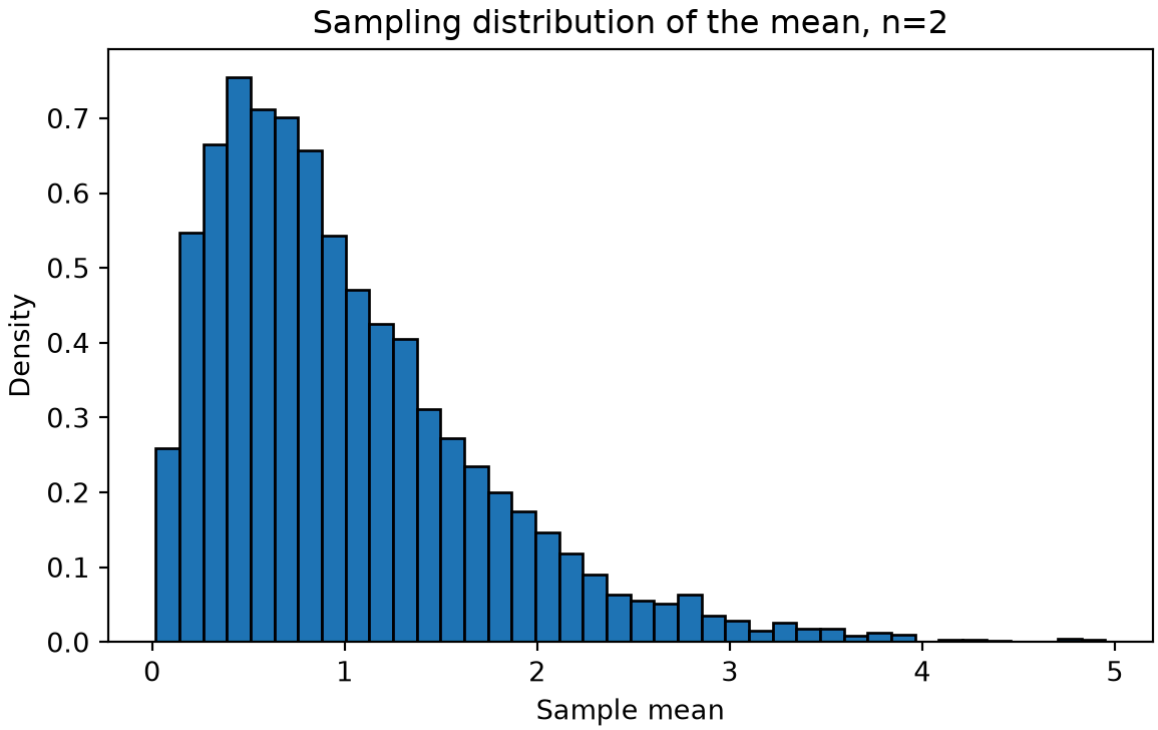


Figure 5: (a) The central limit theorem: sample means become more normal-shaped as sample size increases.

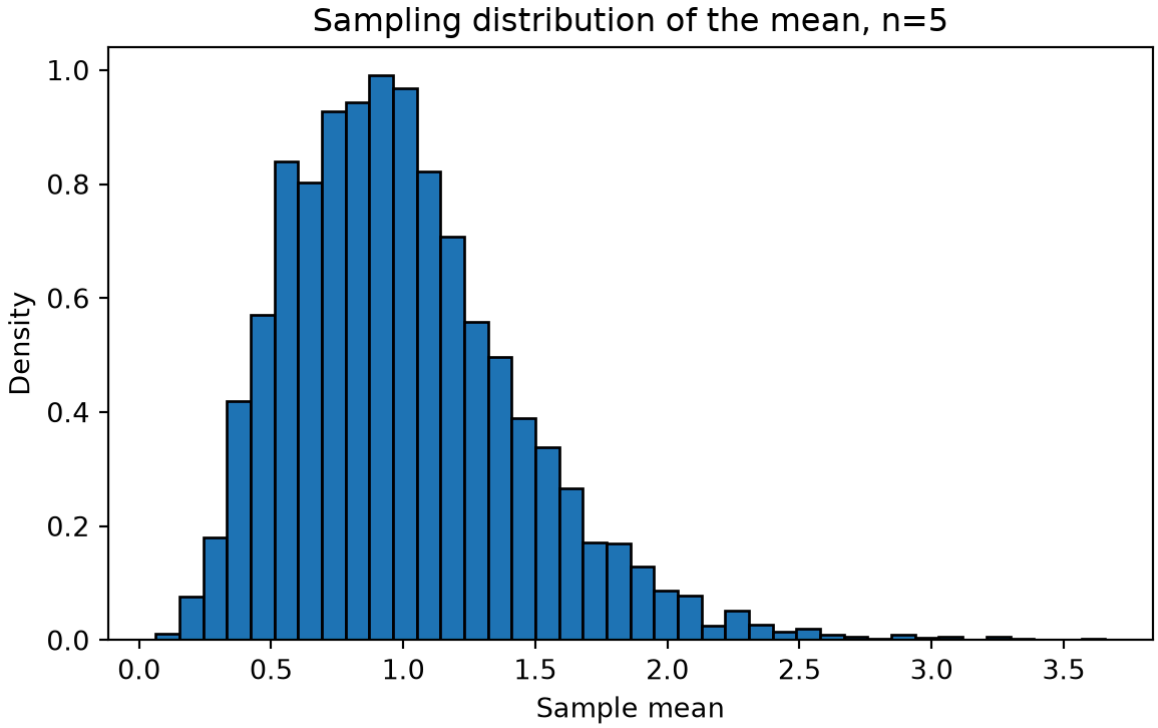
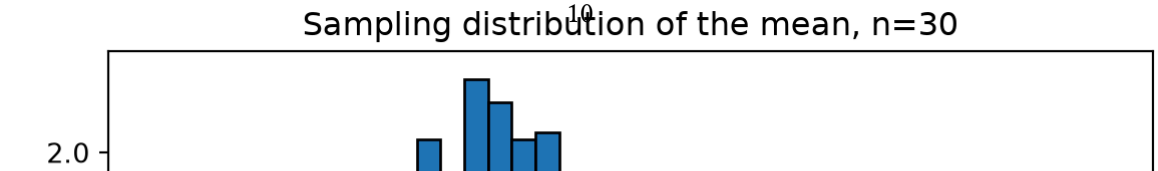


Figure 6: (b)
Figure 4: Figure 4



This theorem is one reason normal-based methods are common. It does not mean all data are normal. It means many averages are approximately normal when sample sizes are large enough and observations are sufficiently independent.

5 Descriptive statistics and exploratory data analysis

Before formal inference, inspect the data. Descriptive **statistics** summarize what was observed; they do not automatically justify conclusions about a broader population.

5.1 A small teaching dataset

The following simulated **dataset** represents students in an introductory course. Each row is one student. The **variables** are:

- `study_hours`: hours studied for a test.
- `attendance`: proportion of lectures attended.
- `review_session`: whether the student attended an optional review session.
- `score`: exam score.

The data are simulated for teaching, not collected from real students.

```
n = 120
study_hours = rng.gamma(shape=3, scale=2, size=n)
attendance = np.clip(rng.normal(0.78, 0.14, size=n), 0.3, 1.0)
review_session = rng.binomial(1, p=0.45, size=n)

# Data-generating process for teaching:
# score depends on study hours, attendance, review session, and random noise.
noise = rng.normal(0, 7, size=n)
score = 55 + 2.4 * study_hours + 12 * attendance + 4.5 * review_session + noise
score = np.clip(score, 0, 100)

students = pd.DataFrame({
    "study_hours": study_hours,
    "attendance": attendance,
    "review_session": review_session,
    "score": score
})

students.head()
```

	study_hours	attendance	review_session	score
0	2.862679	0.748358	0	73.576097
1	7.130522	0.733883	0	78.855770
2	2.783708	0.632852	1	73.972079
3	5.231253	1.000000	0	87.441127
4	6.372933	0.932783	1	86.235048

5.2 Summaries

```
students.describe()
```

	study_hours	attendance	review_session	score
count	120.000000	120.000000	120.000000	120.000000
mean	5.576298	0.785238	0.425000	78.287335
std	3.214783	0.143691	0.496416	9.566944
min	0.918025	0.439859	0.000000	61.494510
25%	3.397686	0.673594	0.000000	71.152046
50%	4.806480	0.795098	0.000000	77.727391
75%	7.210374	0.903067	1.000000	85.238970
max	17.384988	1.000000	1.000000	100.000000

```
students.groupby("review_session")["score"].agg(["count", "mean", "std"])
```

review_session	count	mean	std
0	69	75.670367	7.598096
1	51	81.827939	10.820402

5.3 Visual inspection

```
plt.figure(figsize=(8, 4.5))
plt.hist(students["score"], bins=20, edgecolor="black")
plt.xlabel("Exam score")
plt.ylabel("Number of students")
plt.title("Distribution of exam scores")
plt.show()
```

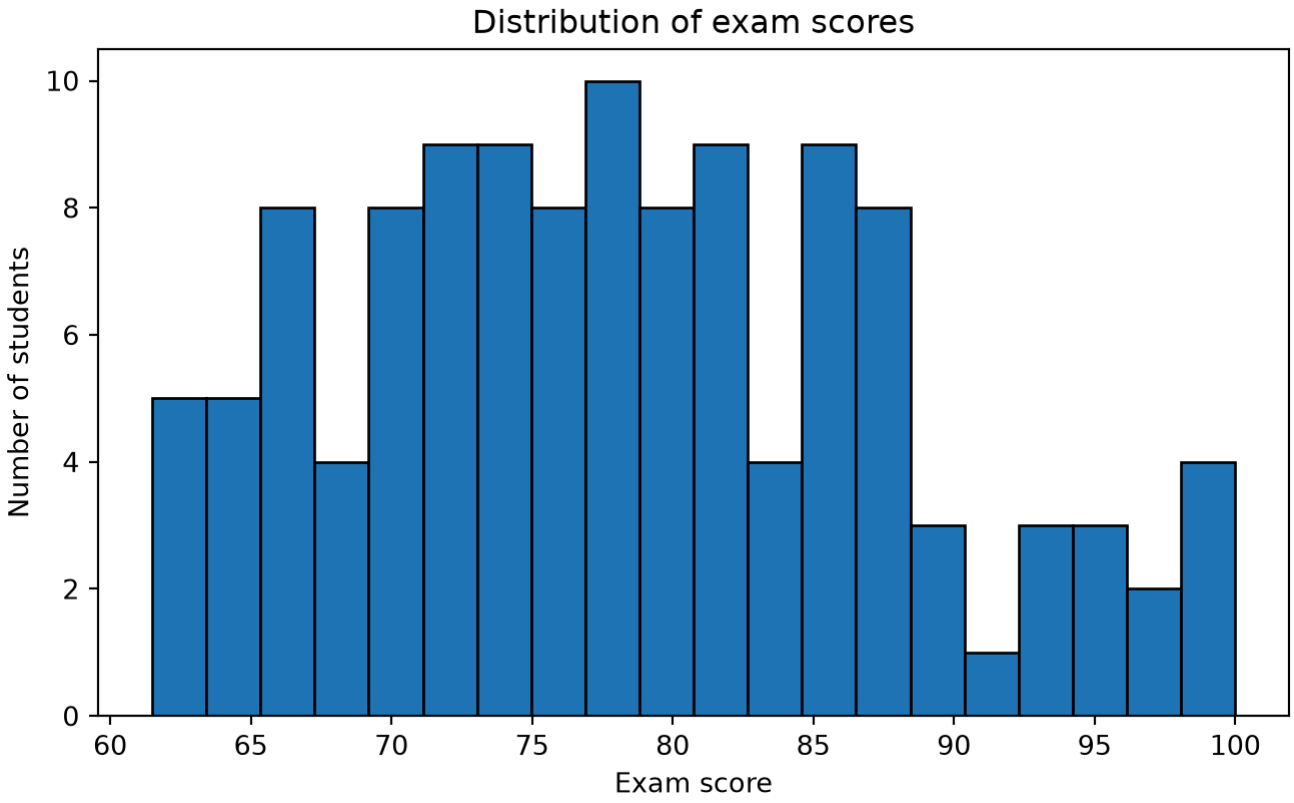


Figure 8: Figure 5: Distribution of exam scores in the teaching dataset.

```
scores_no = students.loc[students["review_session"] == 0, "score"]
scores_yes = students.loc[students["review_session"] == 1, "score"]

plt.figure(figsize=(7, 4.5))
plt.boxplot([scores_no, scores_yes], tick_labels=["No review", "Review"])
plt.ylabel("Exam score")
plt.title("Scores by review-session attendance")
plt.show()
```

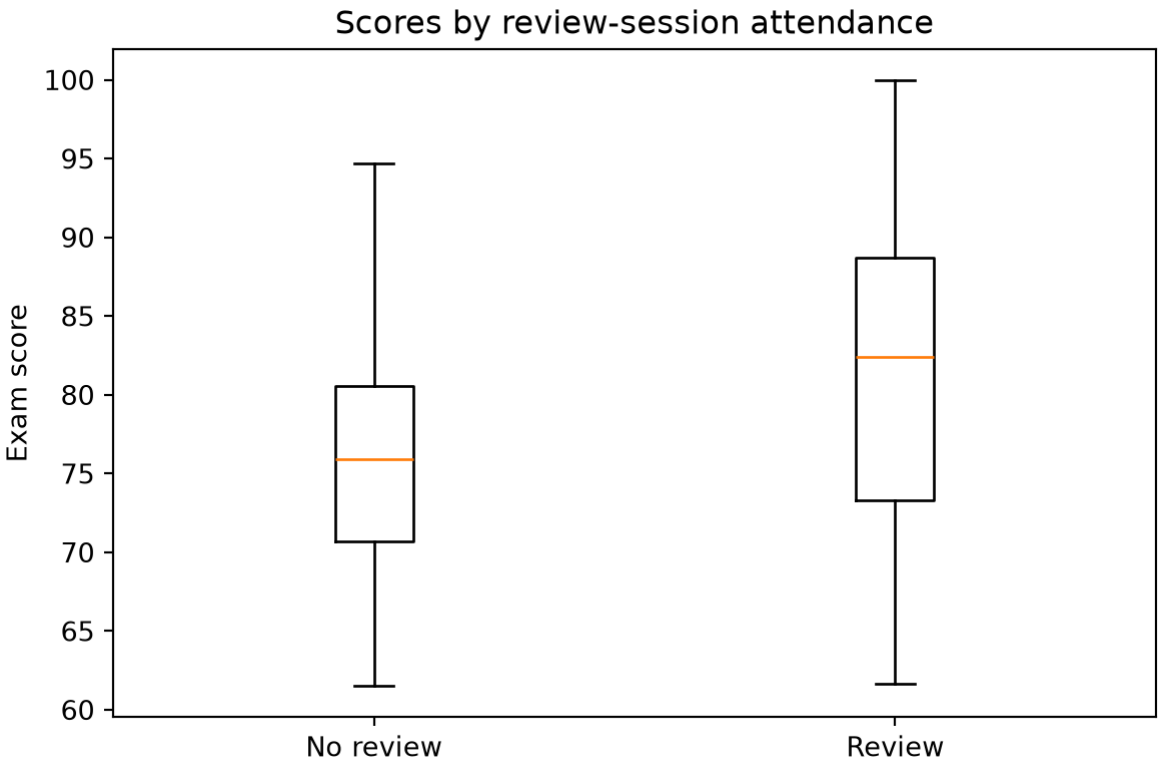


Figure 9: Figure 6: Exam scores by optional review-session attendance.

```
plt.figure(figsize=(8, 4.5))
plt.scatter(students["study_hours"], students["score"], alpha=0.75)
plt.xlabel("Study hours")
plt.ylabel("Exam score")
plt.title("Exam score versus study hours")
plt.show()
```

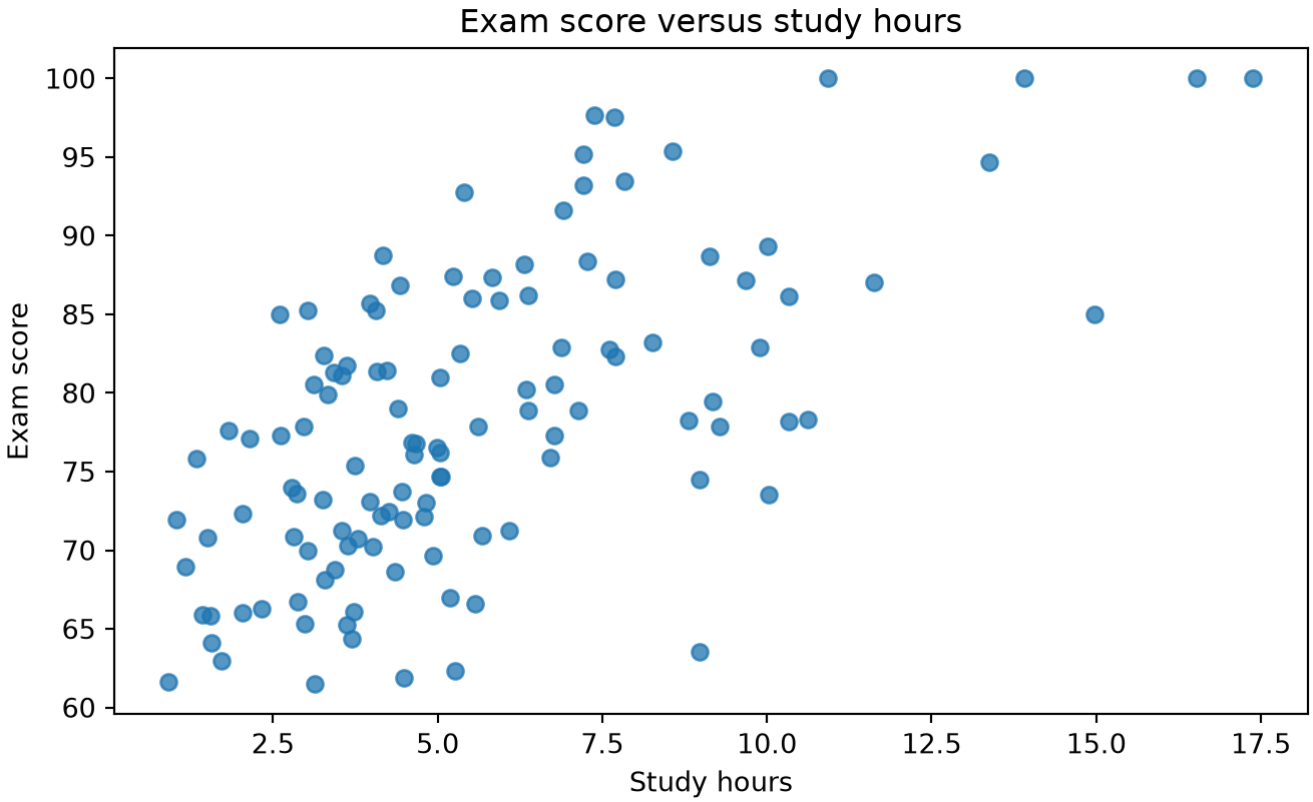


Figure 10: Figure 7: Scatterplot of exam score against study hours.

Exploratory plots help identify patterns, unusual values, and possible modeling choices. They do not by themselves prove that one variable causes another.

6 Estimation and confidence intervals

6.1 Point estimates

A point estimate is a single-number estimate of an unknown parameter. Examples:

- \(\bar{x}\) estimates the population mean \(\mu\).
- \(\hat{p}\) estimates the population proportion \(p\).
- \(\hat{\beta}_1\) estimates the regression slope \(\beta_1\).

For the average exam score:

```
mean_score = students["score"].mean()
mean_score
```

```
np.float64(78.28733504318683)
```

6.2 Confidence intervals

A **confidence interval** gives a range of plausible parameter values under a statistical model.

For a mean, when the population standard deviation is unknown, a common interval is

\[\bar{x} \pm t^*_{n-1}\frac{s}{\sqrt{n}}, \]

where \(t^*_{n-1}\) is a critical value from the Student's \(t\) distribution with \(n-1\) degrees of freedom.

```
x = students["score"]
n = len(x)
xbar = x.mean()
s = x.std(ddof=1)
se = s / np.sqrt(n)
confidence = 0.95
alpha = 1 - confidence
t_star = stats.t.ppf(1 - alpha / 2, df=n - 1)
ci = (xbar - t_star * se, xbar + t_star * se)

pd.DataFrame({
    "quantity": ["mean", "standard error", "t critical value", "CI lower", "CI
upper"],
    "value": [xbar, se, t_star, ci[0], ci[1]]
})
```

	quantity	value
0	mean	78.287335
1	standard error	0.873339
2	t critical value	1.980100
3	CI lower	76.558037
4	CI upper	80.016633

A 95% **confidence interval** does not mean there is a 95% probability that this particular interval contains the true mean, assuming a fixed frequentist parameter. Instead, if the same sampling procedure were repeated many times, about 95% of such intervals would contain the true parameter.

6.3 Confidence intervals by simulation

```
true_mu = 50
true_sigma = 10
n = 25
n_intervals = 80

intervals = []
contains = []
for i in range(n_intervals):
    sample = rng.normal(true_mu, true_sigma, size=n)
    xbar = sample.mean()
    s = sample.std(ddof=1)
    se = s / np.sqrt(n)
```

```
    t_star = stats.t.ppf(0.975, df=n - 1)
    lo = xbar - t_star * se
    hi = xbar + t_star * se
    intervals.append((lo, hi))
    contains.append(lo <= true_mu <= hi)

plt.figure(figsize=(8, 8))
for i, ((lo, hi), ok) in enumerate(zip(intervals, contains)):
    plt.plot([lo, hi], [i, i], linewidth=1)
    plt.plot([(lo + hi) / 2], [i], marker="o", markersize=3)
plt.axvline(true_mu, linestyle="--", linewidth=1)
plt.xlabel("Confidence interval for mean")
plt.ylabel("Simulated sample")
plt.title("Repeated 95% confidence intervals")
plt.show()

sum(contains), n_intervals
```

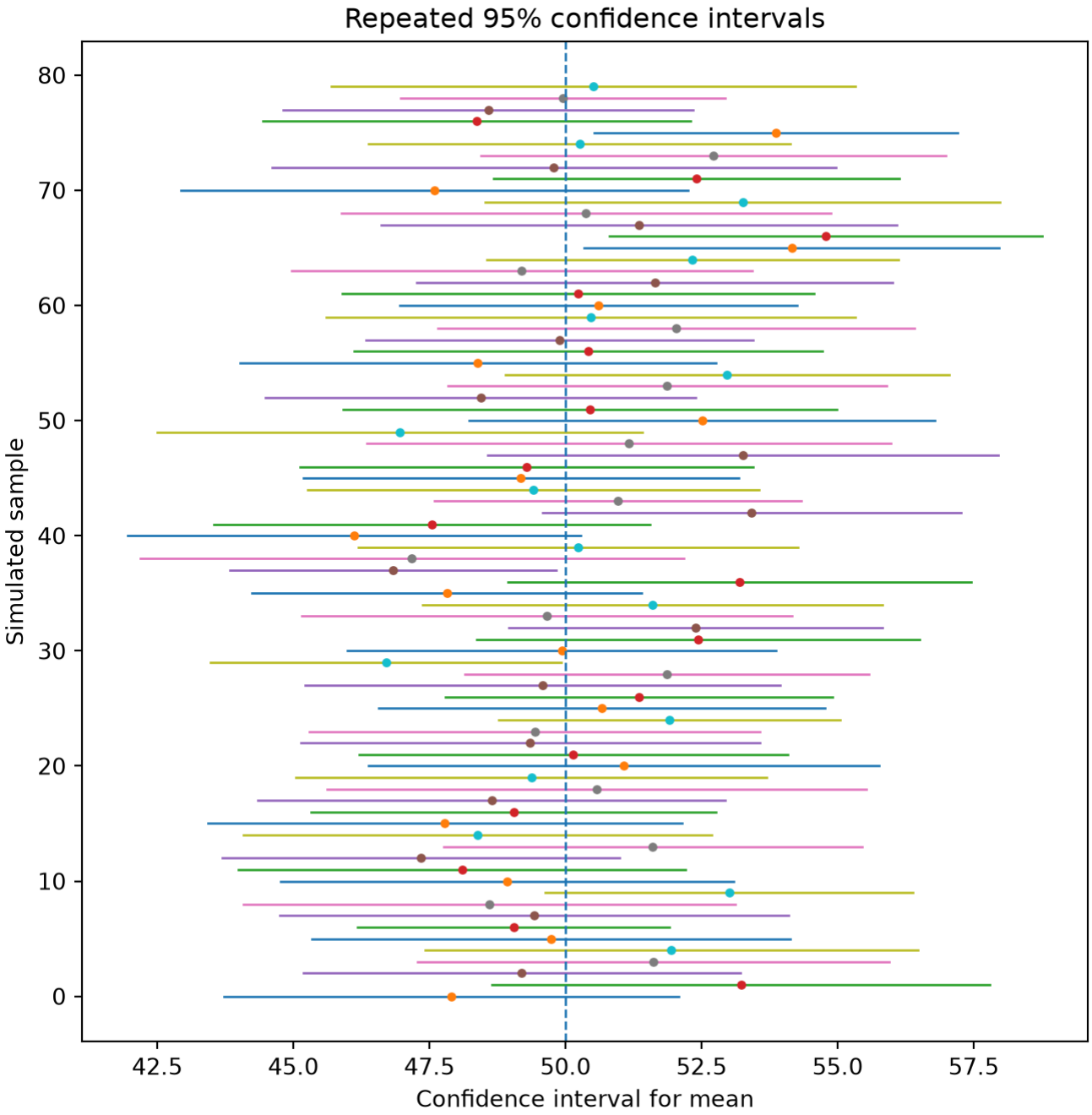


Figure 11: Figure 8: Confidence intervals vary from sample to sample. Most 95% intervals contain the true mean, but some do not.

7 Hypothesis tests and significance tests

7.1 Basic idea

A **hypothesis** test compares observed data to what we would expect under a null hypothesis.

A common workflow is:

```
flowchart TD
  A[State research question] --> B[Define null hypothesis H0]
  B --> C[Define alternative hypothesis H1 or HA]
  C --> D[Choose test statistic]
  D --> E[Compute observed test statistic]
  E --> F[Compute p-value under H0]
  F --> G[Compare p-value with alpha]
  G --> H[Interpret in context]
```


```
flowchart TD
  A[State research question] --> B[Define null hypothesis H0]
  B --> C[Define alternative hypothesis H1 or HA]
  C --> D[Choose test statistic]
  D --> E[Compute observed test statistic]
  E --> F[Compute p-value under H0]
  F --> G[Compare p-value with alpha]
  G --> H[Interpret in context]
```


Listing 2:

The **null hypothesis** \(H_0\) is a baseline claim, often representing no effect, no difference, or a specific parameter value. The **alternative hypothesis** \(H_A\) represents the pattern the researcher is looking for.

Examples:

Question	Null hypothesis	Alternative hypothesis
Is the average score different from 75?	\(\mu=75\)	\(\mu\ne75\)
Is the review group mean higher?	\(\mu_{review}\le\mu_{no\ review}\)	\(\mu_{review}>\mu_{no\ review}\)
Is study time associated with score?	\(\beta_1=0\)	\(\beta_1\ne0\)

7.2 Hypothesis test vs. significance test

The terms are often used interchangeably, but it is useful to separate two related ideas.

A **significance test** usually focuses on the p-value: how surprising are the data if the null **hypothesis** were true?

A **hypothesis test** often includes a pre-specified decision rule: reject \(H_0\) if the p-value is less than a chosen **significance level** \(\alpha\).

For example, with \(\alpha = 0.05\):

- If \(p < 0.05\), reject \(H_0\).
- If \(p \ge 0.05\), do not reject \(H_0\).

The phrase “do not reject” is important. It does not mean we have proven the null **hypothesis** true. It means the data did not provide strong enough evidence against it using the chosen test and **significance level**.

7.3 The p-value

A p-value is the probability, assuming the null **hypothesis** is true, of getting a test statistic as extreme as or more extreme than the one observed.

A p-value is not:

- The probability that the null **hypothesis** is true.
- The probability that the **alternative hypothesis** is true.
- The probability that the result happened by chance.
- A measure of the size or practical importance of an effect.

7.4 Significance level, Type I error, Type II error, and power

The **significance level** \(\alpha\) is the long-run probability of rejecting a true null **hypothesis** when the test assumptions hold. This is called a **Type I error**.

A **Type II error** happens when we fail to reject a false null **hypothesis**.

Power is the probability of rejecting a false null **hypothesis**:

\[\text{Power} = 1 - P(\text{Type II error}). \]

Reality	Decision: do not reject \(H_0\)	Decision: reject \(H_0\)
\(H_0\) true	Correct non-rejection	Type I error
\(H_0\) false	Type II error	Correct rejection

7.5 One-sample t-test

Suppose we want to test whether the population mean exam score differs from 75.

\[H_0: \mu = 75 \]

\[H_A: \mu \ne 75 \]

The one-sample t statistic is

\[t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}. \]

```
mu0 = 75
x = students["score"]
n = len(x)
xbar = x.mean()
s = x.std(ddof=1)
se = s / np.sqrt(n)
t_stat = (xbar - mu0) / se
p_value = 2 * stats.t.sf(abs(t_stat), df=n - 1)

pd.DataFrame({
```

```
    "quantity": ["sample mean", "hypothesized mean", "standard error", "t
statistic", "p-value"],
    "value": [xbar, mu0, se, t_stat, p_value]
})
```

	quantity	value
0	sample mean	78.287335
1	hypothesized mean	75.000000
2	standard error	0.873339
3	t statistic	3.764102
4	p-value	0.000261

The same test can be done with SciPy:

```
stats.ttest_1samp(students["score"], popmean=75)
```

```
TtestResult(statistic=np.float64(3.764101620327317),
pvalue=np.float64(0.0002610704768896567), df=np.int64(119))
```

7.6 Visualizing a two-sided p-value

```
x_grid = np.linspace(-4, 4, 500)
df = len(students) - 1
y_grid = stats.t.pdf(x_grid, df=df)

plt.figure(figsize=(8, 4.5))
plt.plot(x_grid, y_grid)
plt.axvline(t_stat, linestyle="--", linewidth=1)
plt.axvline(-abs(t_stat), linestyle="--", linewidth=1)
plt.axvline(abs(t_stat), linestyle="--", linewidth=1)

# Shade tails
left_tail = x_grid <= -abs(t_stat)
right_tail = x_grid >= abs(t_stat)
plt.fill_between(x_grid[left_tail], y_grid[left_tail], alpha=0.3)
plt.fill_between(x_grid[right_tail], y_grid[right_tail], alpha=0.3)

plt.xlabel("t statistic under the null hypothesis")
plt.ylabel("Density")
plt.title("Two-sided p-value for a one-sample t-test")
plt.show()
```

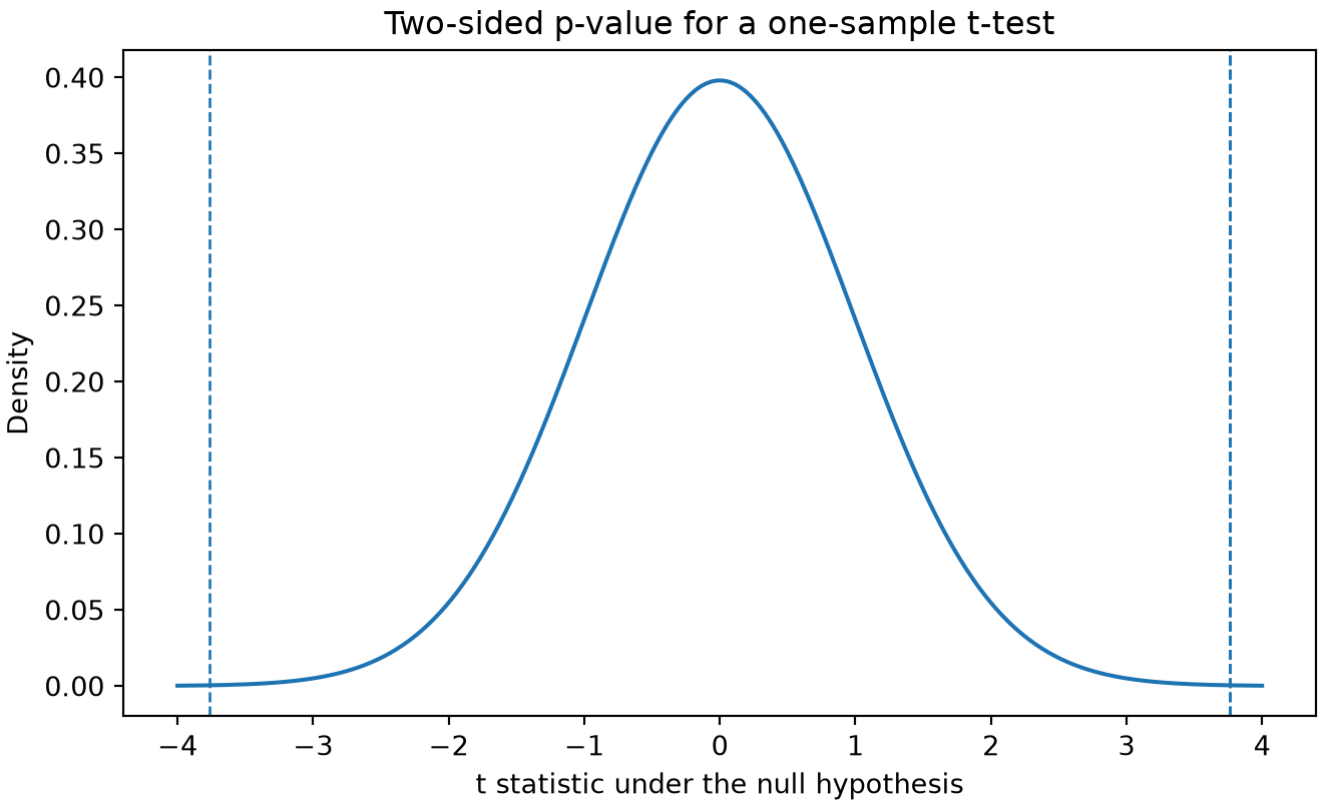


Figure 12: Figure 9: A two-sided p-value is the area in both tails at least as extreme as the observed test statistic.

7.7 Interpreting test results

A good interpretation includes all of the following:

1. The parameter or comparison being tested.
2. The estimate from the data.
3. The p-value or **confidence interval**.
4. A conclusion in context.
5. A warning about assumptions or limitations when relevant.

Template:

> We tested whether the population mean exam score differs from 75. The sample mean is computed in the code above. Under a one-sample t-test, the p-value is computed from the t distribution. At \(\alpha=0.05\), we reject or do not reject the null depending on whether the p-value is below 0.05. This result should be interpreted under the assumptions of independent observations and an approximately valid t-test model.

Quarto's inline Python syntax may depend on the rendering engine configuration. A safer approach in teaching documents is often to compute values in code chunks and write the conclusion manually.

7.8 Two-sample t-test

Now suppose we compare scores for students who attended the optional review session with those who did not.

Let \(\mu_1\) be the population mean score for students who attended the review session, and \(\mu_0\) the population mean score for students who did not.

A two-sided test is:

\[H_0: \mu_1 - \mu_0 = 0 \]

\[H_A: \mu_1 - \mu_0 \ne 0 \]

```
review = students.loc[students["review_session"] == 1, "score"]
no_review = students.loc[students["review_session"] == 0, "score"]

summary = pd.DataFrame({
    "group": ["No review", "Review"],
    "n": [len(no_review), len(review)],
    "mean": [no_review.mean(), review.mean()],
    "sd": [no_review.std(ddof=1), review.std(ddof=1)]
})
summary
```

	group	n	mean	sd
0	No review	69	75.670367	7.598096
1	Review	51	81.827939	10.820402

Welch's t-test does not assume equal population variances.

```
stats.ttest_ind(review, no_review, equal_var=False)
```

```
TtestResult(statistic=np.float64(3.479137071234403),
pvalue=np.float64(0.0007966286697875997), df=np.float64(84.80426876919141))
```

An estimated mean difference is:

```
mean_diff = review.mean() - no_review.mean()
mean_diff
```

```
np.float64(6.157571279001971)
```

For teaching, compare the p-value with the effect size. A tiny p-value for a very small effect may not matter in practice, especially with a large sample. A large p-value can occur when the effect is meaningful but the sample is too small to estimate it precisely.

7.9 Confidence interval for a two-sample mean difference

```
n1, n0 = len(review), len(no_review)
s1, s0 = review.std(ddof=1), no_review.std(ddof=1)
se_diff = np.sqrt(s1**2 / n1 + s0**2 / n0)

# Welch-Satterthwaite degrees of freedom
welch_df = (s1**2/n1 + s0**2/n0)**2 / ((s1**2/n1)**2/(n1-1) + (s0**2/n0)**2/
(n0-1))
t_star = stats.t.ppf(0.975, df=welch_df)
ci_diff = (mean_diff - t_star * se_diff, mean_diff + t_star * se_diff)

pd.DataFrame({
    "quantity": ["mean difference", "standard error", "Welch df", "CI lower", "CI
upper"],
    "value": [mean_diff, se_diff, welch_df, ci_diff[0], ci_diff[1]]
})
```

	quantity	value
0	mean difference	6.157571
1	standard error	1.769856
2	Welch df	84.804269
3	CI lower	2.638506
4	CI upper	9.676636

7.10 One-sided tests

A one-sided alternative tests for a direction.

For example:

\[H_0: \mu_1 - \mu_0 \le 0 \]

\[H_A: \mu_1 - \mu_0 > 0 \]

Use a one-sided test only when the direction was chosen before seeing the data and when an effect in the opposite direction would not support the same conclusion.

```
# SciPy supports alternatives in recent versions.
stats.ttest_ind(review, no_review, equal_var=False, alternative="greater")
```

```
TtestResult(statistic=np.float64(3.479137071234403),
pvalue=np.float64(0.00039831433489379985), df=np.float64(84.80426876919141))
```

7.11 Simulation view of a null distribution

A test statistic is compared with a null distribution. We can often approximate a null distribution by simulation.

For the review-session comparison, the null **hypothesis** says that group labels do not matter. A permutation test repeatedly shuffles the labels and recomputes the mean difference.

```
observed_diff = review.mean() - no_review.mean()
B = 5_000
permuted_diffs = np.empty(B)

scores = students["score"].to_numpy()
labels = students["review_session"].to_numpy()

for b in range(B):
    shuffled = rng.permutation(labels)
    perm_review = scores[shuffled == 1]
    perm_no_review = scores[shuffled == 0]
    permuted_diffs[b] = perm_review.mean() - perm_no_review.mean()

p_perm = np.mean(np.abs(permuted_diffs) >= abs(observed_diff))

plt.figure(figsize=(8, 4.5))
plt.hist(permuted_diffs, bins=40, edgecolor="black")
plt.axvline(observed_diff, linestyle="--", linewidth=1)
plt.axvline(-abs(observed_diff), linestyle="--", linewidth=1)
plt.axvline(abs(observed_diff), linestyle="--", linewidth=1)
plt.xlabel("Mean difference under shuffled labels")
plt.ylabel("Frequency")
plt.title(f"Permutation test null distribution; p ≈ {p_perm:.3f}")
plt.show()

p_perm
```

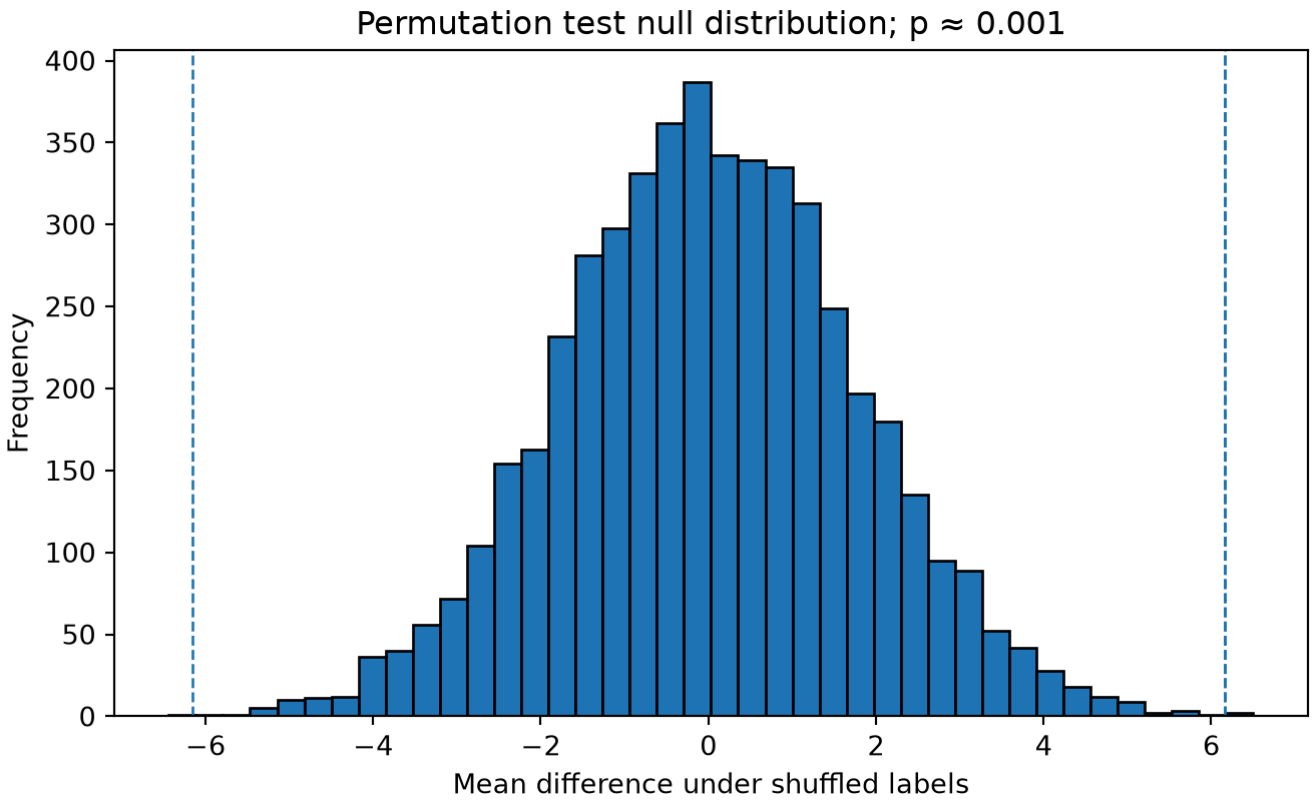


Figure 14: (a) Permutation null distribution for the difference in mean exam score between review and no-review groups.

```
np.float64(0.0006)
```

Listing 3: (b)

Figure 13: Figure 10

Permutation tests are valuable because they show the logic of inference directly: compare the observed statistic with values that would be plausible if the null **hypothesis** were true.

7.12 Multiple testing

If we run many tests, some may be significant by chance even when all null hypotheses are true. With \(m\) independent tests and \(\alpha=0.05\), the expected number of false positives is \(0.05m\).

```
m = 1_000
p_values_under_null = rng.uniform(0, 1, size=m)

plt.figure(figsize=(8, 4.5))
plt.hist(p_values_under_null, bins=20, edgecolor="black")
plt.axvline(0.05, linestyle="--", linewidth=1)
plt.xlabel("p-value")
plt.ylabel("Frequency")
plt.title("P-values when all null hypotheses are true")
```

```
plt.show()
```

```
(p_values_under_null < 0.05).sum()
```

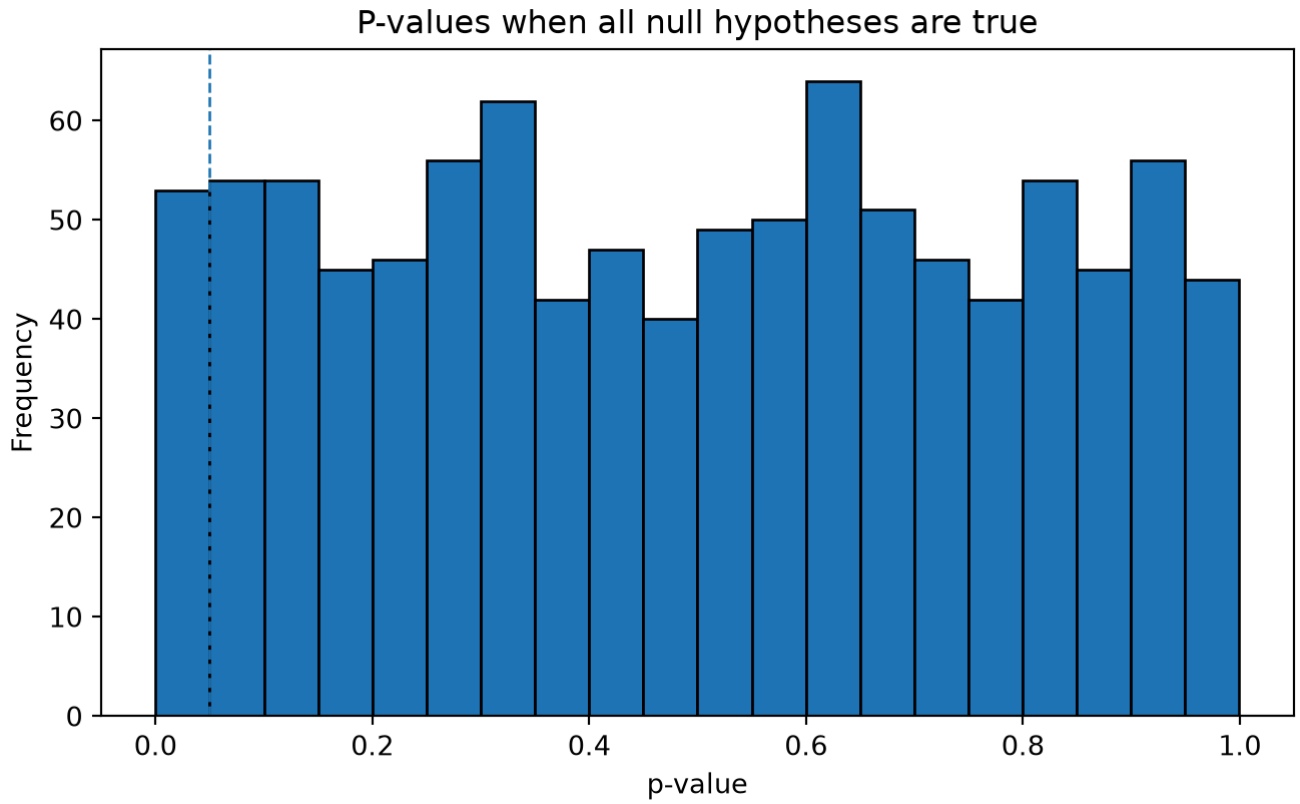


Figure 16: (a) When many null hypotheses are true, some p-values will fall below 0.05 by chance.

```
np.int64(52)
```

Listing 4: (b)

Figure 15: Figure 11

Common responses to multiple testing include reducing the number of tests, pre-specifying hypotheses, reporting all tests, and applying corrections such as Bonferroni or false discovery rate procedures.

8 Simple linear regression

8.1 The regression question

Regression models relationships between **variables**. In **simple linear regression**, we model a numerical outcome \(Y\) using one predictor \(X\).

The population model is:

\[Y_i = \beta_0 + \beta_1X_i + \varepsilon_i. \]

Where:

- \(Y_i\) is the outcome for observation \(i\).
- \(X_i\) is the predictor for observation \(i\).
- \(\beta_0\) is the intercept.
- \(\beta_1\) is the slope.
- \(\varepsilon_i\) is the error term.

The fitted regression line is:

\[\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1X_i. \]

The residual is:

\[e_i = Y_i - \hat{Y}_i. \]

8.2 Least squares

Ordinary least squares chooses the intercept and slope that minimize the sum of squared residuals:

\[\min_{b_0,b_1}\sum_{i=1}^n (y_i - b_0 - b_1x_i)^2. \]

The slope estimate in **simple linear regression** is:

\[\hat{\beta}_1 = \frac{\sum_i (x_i-\bar{x})(y_i-\bar{y})}{\sum_i (x_i-\bar{x})^2}. \]

The intercept estimate is:

\[\hat{\beta}_0 = \bar{y} - \hat{\beta}_1\bar{x}. \]

8.3 Fitting a simple regression manually

We model exam score using study hours.

```
x = students["study_hours"].to_numpy()
y = students["score"].to_numpy()

xbar = x.mean()
ybar = y.mean()

beta1_hat = np.sum((x - xbar) * (y - ybar)) / np.sum((x - xbar) ** 2)
beta0_hat = ybar - beta1_hat * xbar

beta0_hat, beta1_hat
```

```
(np.float64(67.88164434164852), np.float64(1.8660571004540298))
```

Interpretation of the slope: for each additional hour studied, the fitted model predicts an average change of `beta1_hat` points in exam score. The units are “score points per study hour.”

8.4 Fitting the same regression with statsmodels

```
X = sm.add_constant(students["study_hours"])
model = sm.OLS(students["score"], X).fit()
model.summary()
```

Table 13: OLS Regression Results

Dep. Variable:	score	R-squared:	0.393
Model:	OLS	Adj. R-squared:	0.388
Method:	Least Squares	F-statistic:	76.46
Date:	Tue, 30 Jun 2026	Prob (F-statistic):	1.83e-14
Time:	02:49:30	Log-Likelihood:	−410.80
No. Observations:	120	AIC:	825.6
Df Residuals:	118	BIC:	831.2
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	67.8816	1.372	49.470	0.000	65.164	70.599
study_hours	1.8661	0.213	8.744	0.000	1.443	2.289

Omnibus:	0.909	Durbin-Watson:	1.548
Prob(Omnibus):	0.635	Jarque-Bera (JB):	0.912
Skew:	0.031	Prob(JB):	0.634
Kurtosis:	2.577	Cond. No.	13.2

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In the regression output, focus first on:

- `coef`: the estimated intercept and slope.
- `std err`: the standard error of each coefficient estimate.
- `t`: the t statistic for testing whether a coefficient equals zero.
- `P>|t|`: the two-sided p-value for that t test.
- `R-squared`: the proportion of variation in the outcome explained by the model.

8.5 Visualizing the fitted line

```
x_grid = np.linspace(students["study_hours"].min(),
students["study_hours"].max(), 100)
X_grid = sm.add_constant(x_grid)
y_hat_grid = model.predict(X_grid)

plt.figure(figsize=(8, 4.5))
```

```
plt.scatter(students["study_hours"], students["score"], alpha=0.75)
plt.plot(x_grid, y_hat_grid, linewidth=2)
plt.xlabel("Study hours")
plt.ylabel("Exam score")
plt.title("Fitted simple regression line")
plt.show()
```

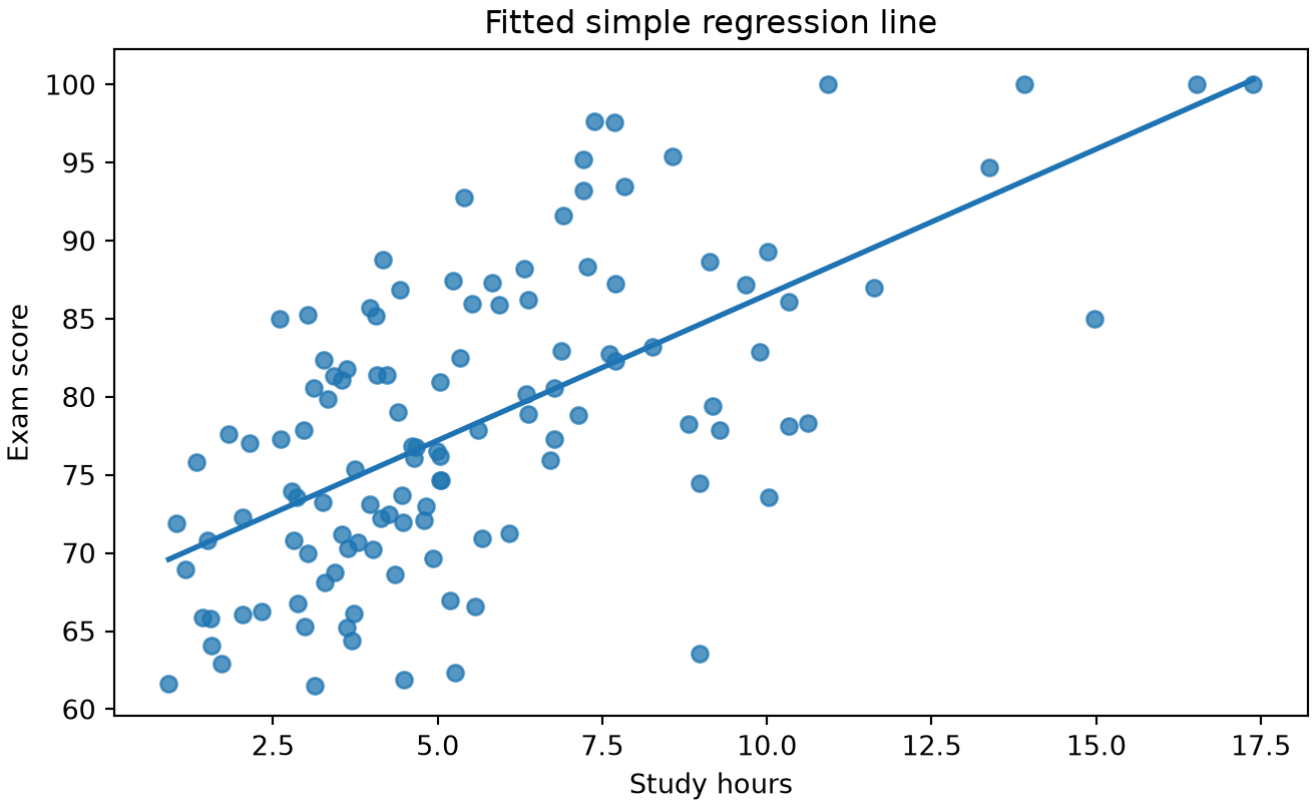


Figure 17: Figure 12: Simple linear regression of exam score on study hours.

8.6 Residuals

Residuals are the vertical differences between observed and fitted values.

```
students = students.copy()
students["fitted_score"] = model.fittedvalues
students["residual"] = model.resid
students[["study_hours", "score", "fitted_score", "residual"]].head()
```

	study_hours	score	fitted_score	residual
0	2.862679	73.576097	73.223568	0.352529
1	7.130522	78.855770	81.187606	−2.331836
2	2.783708	73.972079	73.076203	0.895876
3	5.231253	87.441127	77.643461	9.797666
4	6.372933	86.235048	79.773902	6.461145

```
sample_points = students.sample(8, random_state=1)

plt.figure(figsize=(8, 4.5))
plt.scatter(students["study_hours"], students["score"], alpha=0.35)
plt.plot(x_grid, y_hat_grid, linewidth=2)

for _, row in sample_points.iterrows():
    plt.plot(
        [row["study_hours"], row["study_hours"]],
        [row["score"], row["fitted_score"]],
        linewidth=1
    )

plt.xlabel("Study hours")
plt.ylabel("Exam score")
plt.title("Selected residuals")
plt.show()
```

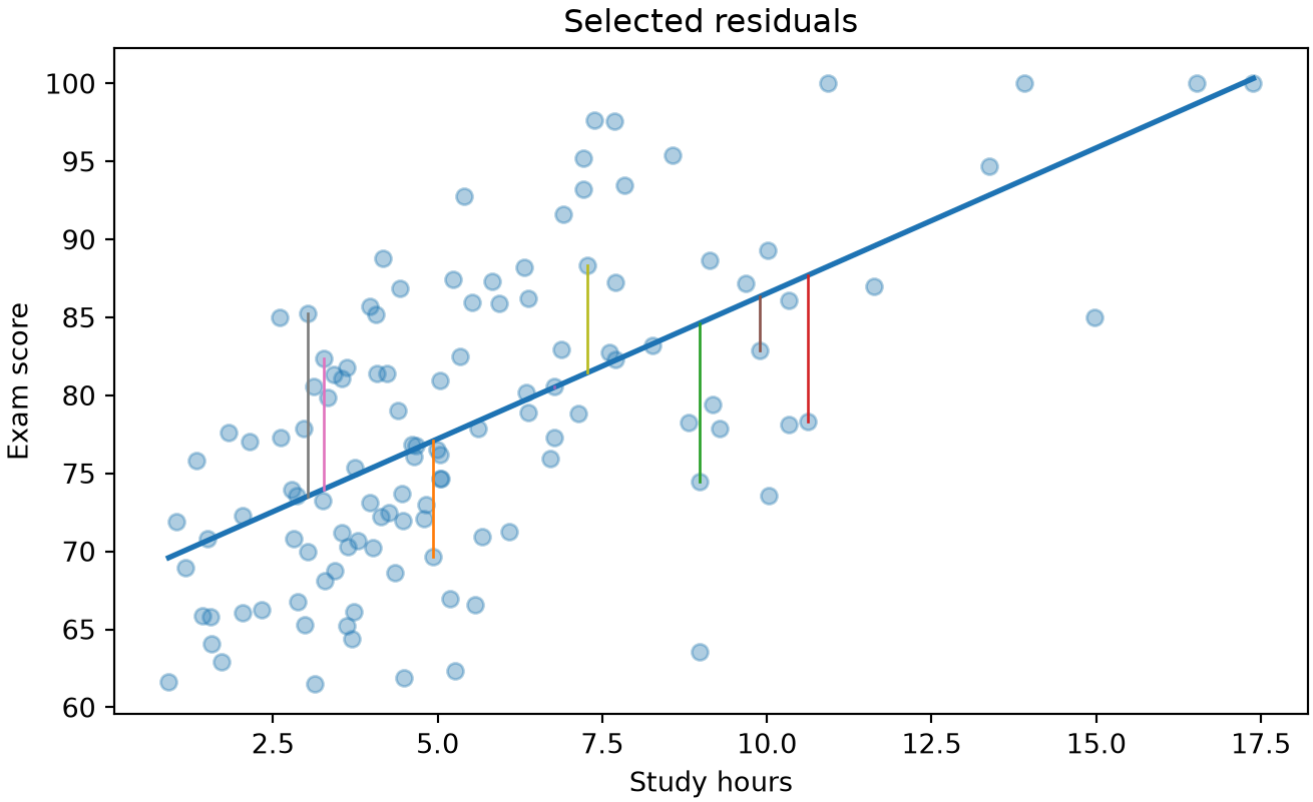


Figure 18: Figure 13: Residuals are vertical distances from observed points to the fitted line.

8.7 Regression inference for the slope

A common **hypothesis** test in simple regression is:

\[H_0: \beta_1 = 0 \]

\[H_A: \beta_1 \ne 0 \]

If \(\beta_1 = 0\), the model says there is no linear association between \(X\) and the expected value of \(Y\).

The t statistic is:

\[t = \frac{\hat{\beta}_1 - 0}{SE(\hat{\beta}_1)}. \]

```
model.params
```

```
const          67.881644
study_hours     1.866057
dtype: float64
```

```
model.bse
```

```
const          1.372179
study_hours    0.213405
dtype: float64
```

```
model.pvalues
```

```
const          9.489645e-81
study_hours    1.829390e-14
dtype: float64
```

```
model.conf_int(alpha=0.05)
```

	0	1
const	65.164356	70.598933
study_hours	1.443456	2.288658

The p-value for the slope tests evidence against a zero linear relationship. It does not prove that study hours cause higher scores. Students who study more may also differ in preparation, motivation, prior knowledge, attendance, or other **variables**.

8.8 Prediction

Regression can be used for prediction. For example, predict the expected score for a student who studied 6 hours.

```
new_data = pd.DataFrame({"const": [1], "study_hours": [6]})
model.predict(new_data)
```

```
0    79.077987
dtype: float64
```

Statsmodels can also produce **confidence intervals** for the expected mean response and prediction intervals for individual outcomes.

```
new_data = sm.add_constant(pd.DataFrame({"study_hours": [2, 4, 6, 8, 10]}),
has_constant="add")
pred = model.get_prediction(new_data).summary_frame(alpha=0.05)
pred
```

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	71.613759	1.024316	69.585335	73.642182	56.655336	86.572181
1	75.345873	0.761514	73.837867	76.853879	60.449095	90.242651
2	79.077987	0.689145	77.713291	80.442682	64.195033	93.960940
3	82.810101	0.856898	81.113210	84.506992	67.893019	97.727184
4	86.542215	1.165316	84.234572	88.849858	71.543378	101.541053

A **confidence interval** for the mean response is narrower than a prediction interval for a new individual observation. Predicting an individual outcome is harder than estimating an average outcome.

8.9 Regression assumptions

Simple linear regression is often introduced with these assumptions:

1. **Linearity**: the expected value of \(Y\) is a linear function of \(X\).
2. **Independence**: observations are independent.
3. **Constant variance**: **errors** have the same variance across values of \(X\).
4. **Approximately normal errors**: mainly important for small-sample inference.
5. **No extreme influential points**: a few observations should not dominate the fitted line.

These assumptions are about the **data-generating process** and the model **errors**, not simply about the observed raw outcome variable.

8.10 Diagnostic plots

```
plt.figure(figsize=(8, 4.5))
plt.scatter(model.fittedvalues, model.resid, alpha=0.75)
plt.axhline(0, linestyle="--", linewidth=1)
plt.xlabel("Fitted score")
plt.ylabel("Residual")
plt.title("Residuals versus fitted values")
plt.show()
```

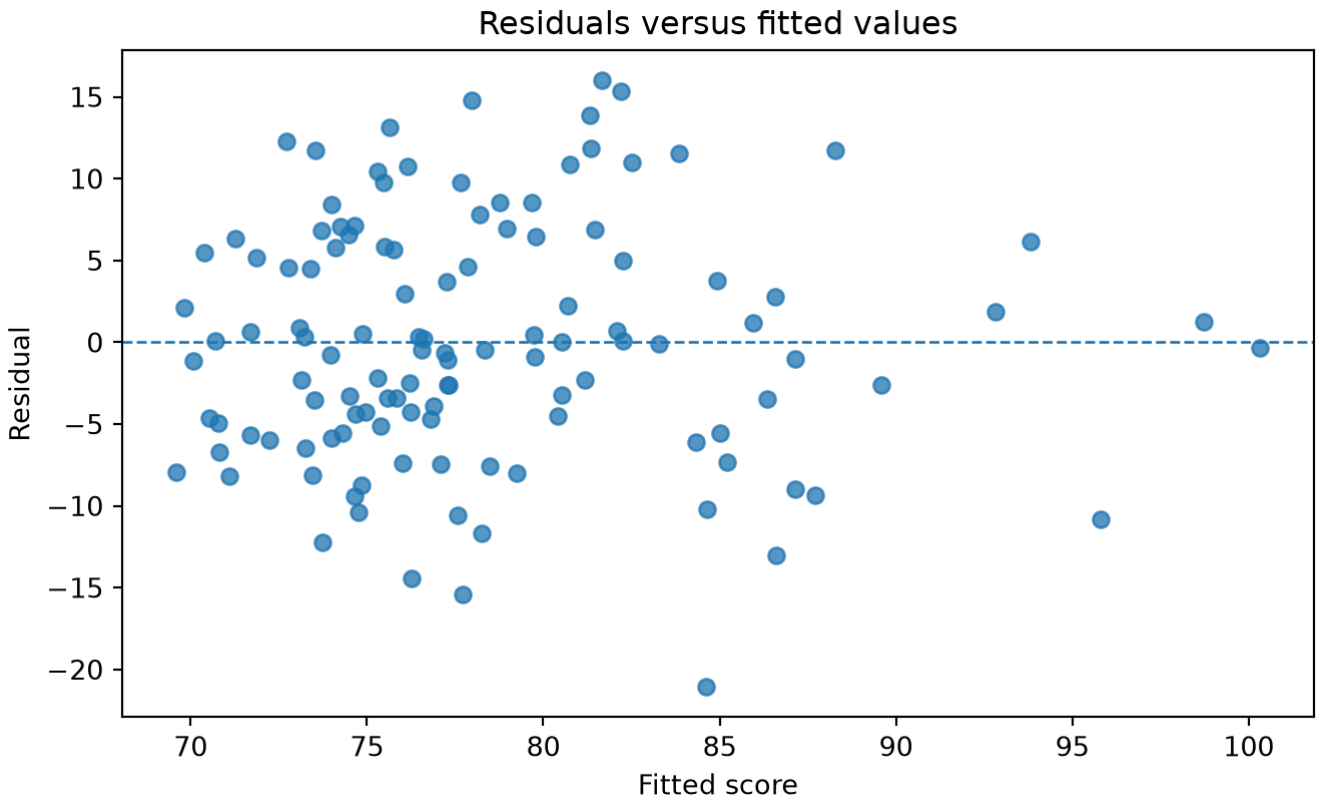


Figure 19: Figure 14: Residuals versus fitted values. Look for curvature, changing spread, and unusual points.

```
sm.qqplot(model.resid, line="45", fit=True)
plt.title("Normal Q-Q plot of residuals")
plt.show()
```

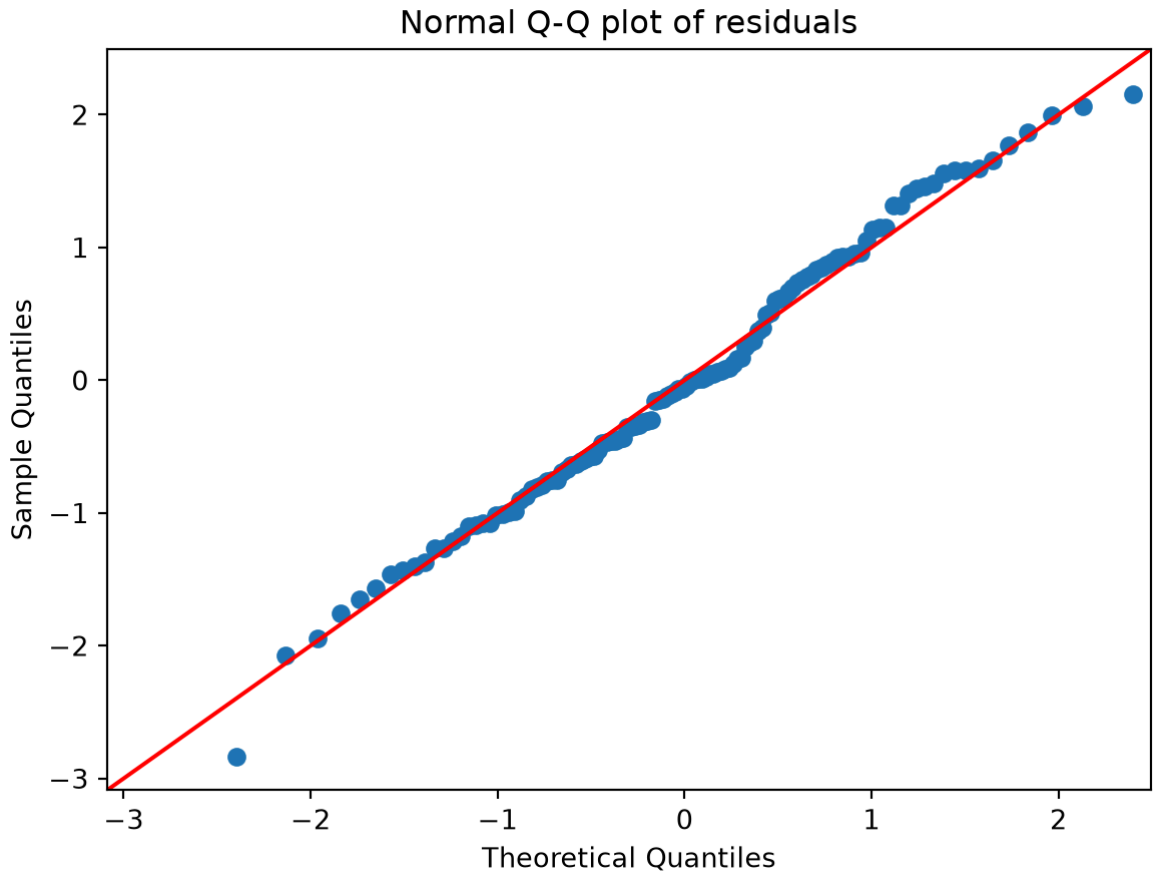


Figure 20: Figure 15: Normal Q-Q plot of residuals. Large systematic departures from the line suggest non-normal errors.

8.11 Outliers and influence

Outliers in \(Y\) and unusual values in \(X\) can affect regression. An observation with an unusual \(X\) value has high leverage. A high-leverage observation can strongly influence the fitted slope if its \(Y\) value does not follow the general pattern.

```
x_base = rng.uniform(0, 10, size=40)
y_base = 10 + 2 * x_base + rng.normal(0, 2, size=40)

# Add one influential point
x_infl = np.append(x_base, 18)
y_infl = np.append(y_base, 12)

# Fit models
X_base = sm.add_constant(x_base)
X_infl = sm.add_constant(x_infl)
```

```
mod_base = sm.OLS(y_base, X_base).fit()
mod_infl = sm.OLS(y_infl, X_infl).fit()

grid = np.linspace(0, 18, 100)

plt.figure(figsize=(8, 4.5))
plt.scatter(x_base, y_base, alpha=0.75, label="Original data")
plt.scatter([18], [12], marker="x", s=80, label="Influential point")
plt.plot(grid, mod_base.predict(sm.add_constant(grid)), linewidth=2,
label="Without point")
plt.plot(grid, mod_infl.predict(sm.add_constant(grid)), linewidth=2,
linestyle="--", label="With point")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Influence in simple regression")
plt.legend()
plt.show()
```

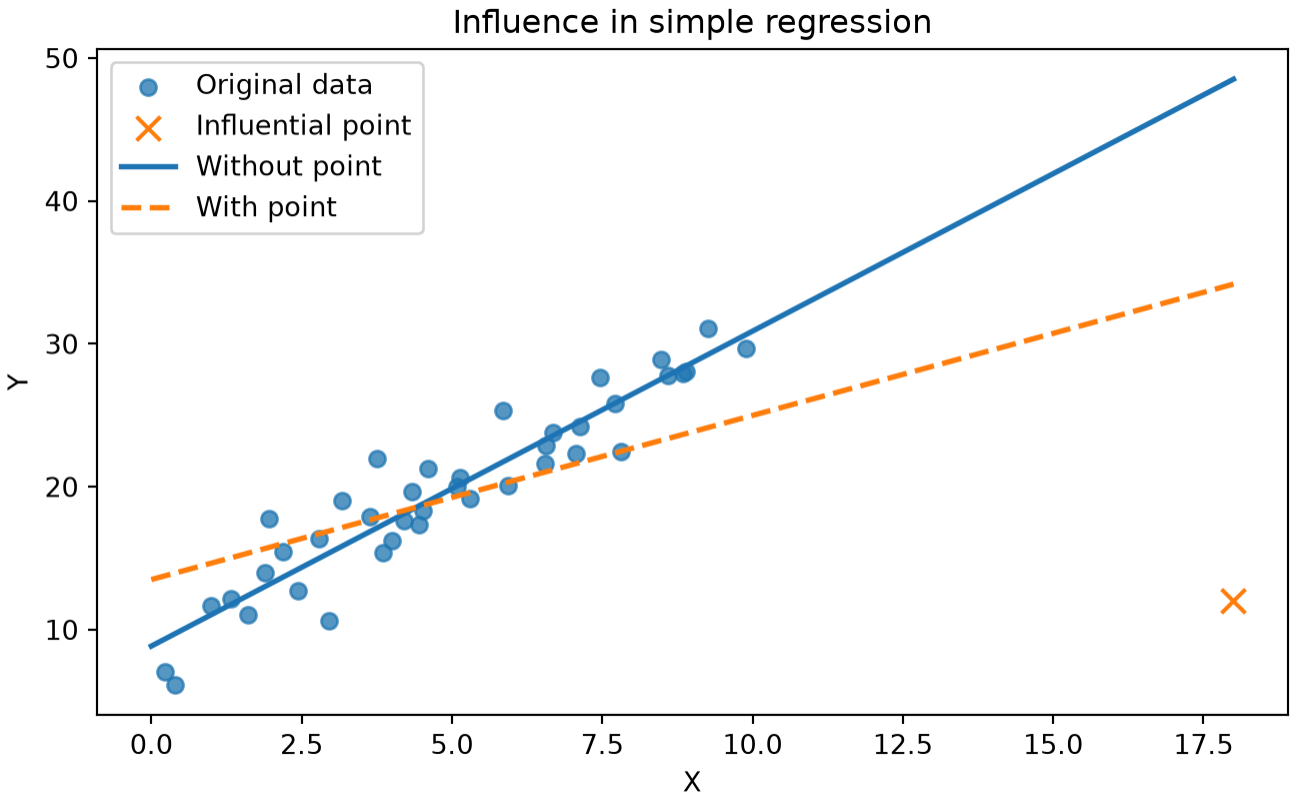


Figure 21: Figure 16: A single high-leverage point can substantially change a regression line.

8.12 Correlation and simple regression

Correlation measures the strength and direction of a linear association between two numerical **variables**:

\[r = \frac{\sum_i (x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_i (x_i-\bar{x})^2\sum_i (y_i-\bar{y})^2}}. \]

Correlation is unitless and lies between −1 and 1. Regression slope has units: units of \(Y\) per one unit of \(X\).

```
r = students[["study_hours", "score"]].corr().iloc[0, 1]
r
```

```
np.float64(0.6270516845121317)
```

For **simple linear regression** with an intercept,

\[R^2 = r^2. \]

```
r**2, model.rsquared
```

```
(np.float64(0.3931938150495019), np.float64(0.393193815049502))
```

8.13 Regression with a binary predictor

A two-sample comparison can be written as a regression with a 0/1 predictor.

```
X_review = sm.add_constant(students["review_session"])
model_review = sm.OLS(students["score"], X_review).fit()
model_review.summary()
```

Table 19: OLS Regression Results

Dep. Variable:	score	R-squared:	0.102
Model:	OLS	Adj. R-squared:	0.094
Method:	Least Squares	F-statistic:	13.42
Date:	Tue, 30 Jun 2026	Prob (F-statistic):	0.000375
Time:	02:49:31	Log-Likelihood:	−434.31
No. Observations:	120	AIC:	872.6
Df Residuals:	118	BIC:	878.2
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	75.6704	1.096	69.044	0.000	73.500	77.841
review_session	6.1576	1.681	3.663	0.000	2.828	9.487

Omnibus:	1.901	Durbin-Watson:	1.555
Prob(Omnibus):	0.387	Jarque-Bera (JB):	1.457
Skew:	0.052	Prob(JB):	0.483
Kurtosis:	2.470	Cond. No.	2.48

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Here, the intercept is the mean score for the group coded 0, and the slope is the difference between the group coded 1 and the group coded 0.

```
students.groupby("review_session")["score"].mean()
```

```
review_session
0    75.670367
1    81.827939
Name: score, dtype: float64
```

```
model_review.params
```

```
const             75.670367
review_session     6.157571
dtype: float64
```

This connection helps unify regression and **hypothesis** testing. A t-test comparing two means and a regression with one binary predictor answer closely related questions.

9 Case study: studying, review sessions, and exam scores

9.1 Research questions

For the teaching **dataset**, suppose we ask:

1. Is the average exam score different from 75?
2. Do students who attended the review session have different average scores from those who did not?
3. Is study time linearly associated with exam score?

9.2 Question 1: one-sample mean test

```
q1 = stats.ttest_1samp(students["score"], popmean=75)
q1
```

```
TtestResult(statistic=np.float64(3.764101620327317),
pvalue=np.float64(0.0002610704768896567), df=np.int64(119))
```

```
q1_summary = pd.DataFrame({
    "estimate": [students["score"].mean()],
    "hypothesized_mean": [75],
```

```
    "test_statistic": [q1.statistic],
    "p_value": [q1.pvalue]
})
q1_summary
```

	estimate	hypothesized_mean	test_statistic	p_value
0	78.287335	75	3.764102	0.000261

Interpretation template:

> The sample mean exam score is shown in the table. The one-sample t-test compares this estimate with 75. If the p-value is less than the chosen **significance level**, we reject the null **hypothesis** that the population mean is 75. The conclusion should be stated as evidence about the population mean, not as proof about every student.

9.3 Question 2: review-session comparison

```
q2 = stats.ttest_ind(review, no_review, equal_var=False)
q2
```

```
TtestResult(statistic=np.float64(3.479137071234403),
pvalue=np.float64(0.0007966286697875997), df=np.float64(84.80426876919141))
```

```
q2_summary = pd.DataFrame({
    "mean_review": [review.mean()],
    "mean_no_review": [no_review.mean()],
    "mean_difference": [review.mean() - no_review.mean()],
    "test_statistic": [q2.statistic],
    "p_value": [q2.pvalue]
})
q2_summary
```

	mean_review	mean_no_review	mean_difference	test_statistic	p_value
0	81.827939	75.670367	6.157571	3.479137	0.000797

Interpretation template:

> The estimated difference in average scores is the review group mean minus the no-review group mean. Welch's t-test assesses whether this observed difference would be unusual if the population means were equal. Because this is observational in the simulated story, a significant difference should not automatically be interpreted as causal.

9.4 Question 3: regression of score on study hours

```
q3_model = sm.OLS(students["score"],
sm.add_constant(students["study_hours"])).fit()
q3_model.summary()
```

Table 24: OLS Regression Results

Dep. Variable:	score	R-squared:	0.393
Model:	OLS	Adj. R-squared:	0.388
Method:	Least Squares	F-statistic:	76.46
Date:	Tue, 30 Jun 2026	Prob (F-statistic):	1.83e-14
Time:	02:49:31	Log-Likelihood:	−410.80
No. Observations:	120	AIC:	825.6
Df Residuals:	118	BIC:	831.2
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	67.8816	1.372	49.470	0.000	65.164	70.599
study_hours	1.8661	0.213	8.744	0.000	1.443	2.289

Omnibus:	0.909	Durbin-Watson:	1.548
Prob(Omnibus):	0.635	Jarque-Bera (JB):	0.912
Skew:	0.031	Prob(JB):	0.634
Kurtosis:	2.577	Cond. No.	13.2

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpretation template:

> The slope estimates the expected difference in exam score associated with one additional study hour. The p-value for the slope tests whether the population linear slope is zero. The **confidence interval** gives a range of plausible slope values. The model is useful only to the extent that the linear form and other assumptions are reasonable.

10 Practical significance, statistical significance, and causality

10.1 Statistical significance

A statistically significant result means the observed data would be relatively unusual under the null **hypothesis**, according to the test model and chosen \(\alpha\).

10.2 Practical significance

Practical significance asks whether the effect is large enough to matter in context.

For example, a 0.2-point increase in exam score may be statistically significant in a **dataset** with hundreds of thousands of students, but it may not matter educationally. A 5-point increase may matter in practice even if a small study lacks enough power to make it statistically significant.

10.3 Causality

Regression and t-tests measure associations unless the study design supports causal interpretation. Stronger causal claims usually require randomized experiments, natural experiments, or careful causal designs.

In the teaching **dataset**, students were not randomly assigned to study hours or review sessions. Therefore, differences may reflect confounding **variables** such as prior preparation or motivation.

11 Common mistakes and better alternatives

Mistake	Better practice
"The p-value is the probability that \(H_0\) is true."	Say: the p-value is computed assuming \(H_0\) is true.
"Fail to reject means \(H_0\) is proven."	Say: the data did not provide strong evidence against \(H_0\).
"Statistically significant means important."	Report effect sizes and context.
"Correlation proves causation."	Discuss study design and possible confounders.
"The regression line is valid everywhere."	Avoid extrapolation beyond the data range.
"Model assumptions are optional."	Check assumptions and explain limitations.
"A non-significant result means no effect."	Consider sample size, uncertainty, and power.

12 Practice exercises

12.1 Exercise 1: probability simulation

Simulate 10,000 samples of 50 coin flips from a fair coin. For each sample, compute the proportion of heads. Answer:

1. What is the mean of the simulated sample proportions?
2. What is their standard deviation?
3. How does this compare with the theoretical standard error \(\sqrt{p(1-p)/n}\)?

```
# Write your code here.
```

12.2 Exercise 2: confidence interval for a mean

Using the `students` **dataset**, compute a 90% **confidence interval** for the mean exam score. Compare it with the 95% interval. Which is wider, and why?

```
# Write your code here.
```

12.3 Exercise 3: one-sample test

Test whether the average attendance proportion differs from 0.80.

1. State \(H_0\) and \(H_A\).
2. Compute the test statistic and p-value.
3. Write a conclusion in context.

```
# Write your code here.
```

12.4 Exercise 4: two-sample test

Compare study hours between students who attended the review session and those who did not.

1. Make a **visualization**.
2. Compute group means.
3. Run Welch's t-test.
4. Explain whether a difference in study hours might affect the interpretation of the review-session score comparison.

```
# Write your code here.
```

12.5 Exercise 5: simple regression

Fit a **simple linear regression** predicting score from attendance.

1. Interpret the intercept and slope.
2. Report the p-value for the slope.
3. Make a scatterplot with the fitted line.
4. Check residuals versus fitted values.

```
# Write your code here.
```

12.6 Exercise 6: prediction and extrapolation

Use the regression of score on study hours to predict scores for students who studied 1, 5, 10, and 20 hours.

1. Which predictions are within the observed range of study hours?
2. Which predictions require extrapolation?
3. Why is extrapolation risky?

```
# Write your code here.
```

13 Selected solutions

13.1 Solution 1

```
B = 10_000
n = 50
p = 0.5
props = rng.binomial(n=1, p=p, size=(B, n)).mean(axis=1)
```

```
sim_mean = props.mean()
sim_sd = props.std(ddof=1)
theory_se = np.sqrt(p * (1 - p) / n)

pd.DataFrame({
    "quantity": ["simulated mean", "simulated SD", "theoretical SE"],
    "value": [sim_mean, sim_sd, theory_se]
})
```

	quantity	value
0	simulated mean	0.499864
1	simulated SD	0.070747
2	theoretical SE	0.070711

13.2 Solution 2

```
def mean_ci(x, confidence=0.95):
    x = np.asarray(x)
    n = len(x)
    xbar = x.mean()
    s = x.std(ddof=1)
    se = s / np.sqrt(n)
    alpha = 1 - confidence
    t_star = stats.t.ppf(1 - alpha / 2, df=n - 1)
    return xbar - t_star * se, xbar + t_star * se

ci90 = mean_ci(students["score"], confidence=0.90)
ci95 = mean_ci(students["score"], confidence=0.95)

pd.DataFrame({
    "confidence": [0.90, 0.95],
    "lower": [ci90[0], ci95[0]],
    "upper": [ci90[1], ci95[1]],
    "width": [ci90[1] - ci90[0], ci95[1] - ci95[0]]
})
```

	confidence	lower	upper	width
0	0.90	76.839550	79.735120	2.895570
1	0.95	76.558037	80.016633	3.458595

The 95% interval is wider because it uses a larger critical value. More confidence requires allowing a wider range of plausible parameter values.

13.3 Solution 3

```
stats.ttest_1samp(students["attendance"], popmean=0.80)
```

```
TtestResult(statistic=np.float64(-1.1254325883068843),
pvalue=np.float64(0.26267049769771295), df=np.int64(119))
```

A two-sided test uses:

\[H_0: \mu_{attendance}=0.80 \]

\[H_A: \mu_{attendance}\ne0.80 \]

Interpret the result using the p-value and the sample mean.

```
students["attendance"].mean()
```

```
np.float64(0.7852375866597823)
```

13.4 Solution 4

```
study_review = students.loc[students["review_session"] == 1, "study_hours"]
study_no_review = students.loc[students["review_session"] == 0, "study_hours"]

plt.figure(figsize=(7, 4.5))
plt.boxplot([study_no_review, study_review], tick_labels=["No review", "Review"])
plt.ylabel("Study hours")
plt.title("Study hours by review-session attendance")
plt.show()

pd.DataFrame({
    "group": ["No review", "Review"],
    "mean_study_hours": [study_no_review.mean(), study_review.mean()],
    "sd_study_hours": [study_no_review.std(ddof=1), study_review.std(ddof=1)]
})
```

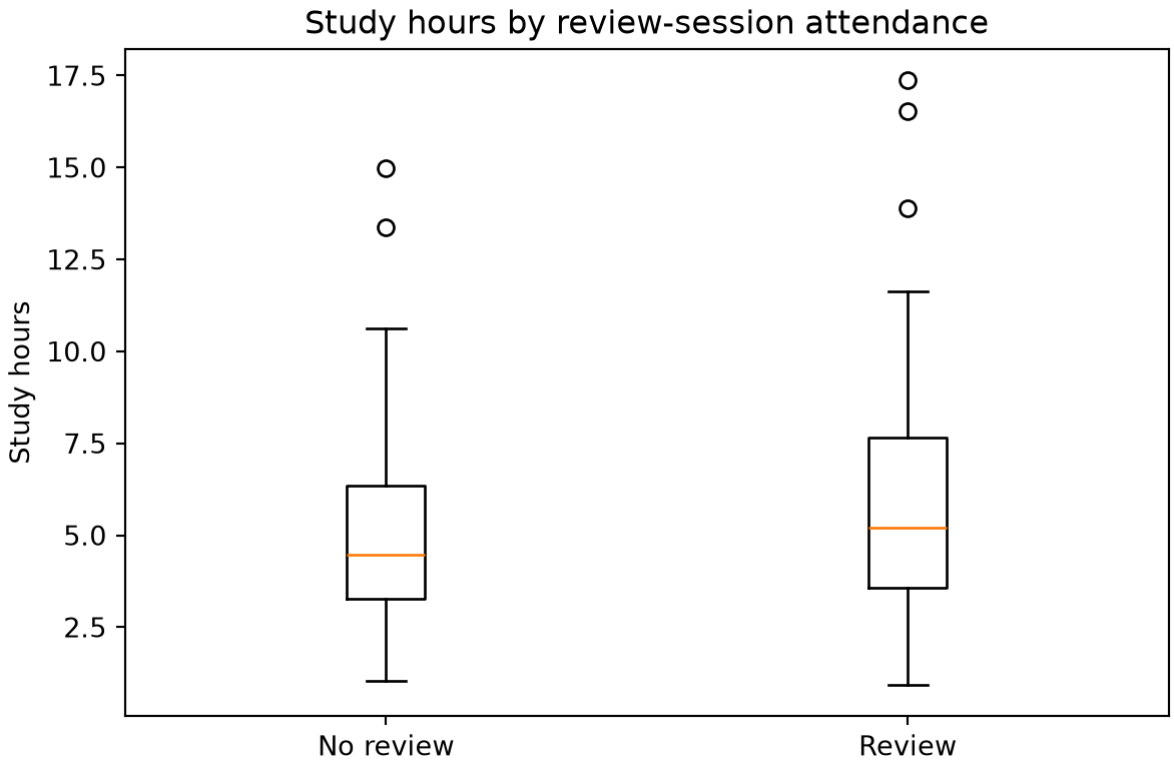


Figure 22:

	group	mean_study_hours	sd_study_hours
0	No review	5.125439	2.755673
1	Review	6.186284	3.689088

```
stats.ttest_ind(study_review, study_no_review, equal_var=False)
```

```
TtestResult(statistic=np.float64(1.7279687619708528),
pvalue=np.float64(0.08747550013041099), df=np.float64(88.65824612087795))
```

If review-session attendees also studied more, then the simple review/no-review score comparison may mix the association with review attendance and the association with study time.

13.5 Solution 5

```
attendance_model = sm.OLS(students["score"],
sm.add_constant(students["attendance"])).fit()
attendance_model.summary()
```

Table 31: OLS Regression Results

Dep. Variable:	score	R-squared:	0.055
Model:	OLS	Adj. R-squared:	0.047
Method:	Least Squares	F-statistic:	6.854
Date:	Tue, 30 Jun 2026	Prob (F-statistic):	0.0100
Time:	02:49:31	Log-Likelihood:	−437.38
No. Observations:	120	AIC:	878.8
Df Residuals:	118	BIC:	884.3
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	66.0377	4.756	13.885	0.000	56.620	75.456
attendance	15.6000	5.959	2.618	0.010	3.800	27.400

Omnibus:	4.722	Durbin-Watson:	1.651
Prob(Omnibus):	0.094	Jarque-Bera (JB):	4.791
Skew:	0.466	Prob(JB):	0.0911
Kurtosis:	2.701	Cond. No.	11.4

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
x_grid = np.linspace(students["attendance"].min(), students["attendance"].max(),
100)
y_grid = attendance_model.predict(sm.add_constant(x_grid))

plt.figure(figsize=(8, 4.5))
plt.scatter(students["attendance"], students["score"], alpha=0.75)
plt.plot(x_grid, y_grid, linewidth=2)
plt.xlabel("Attendance proportion")
plt.ylabel("Exam score")
plt.title("Regression of score on attendance")
plt.show()
```

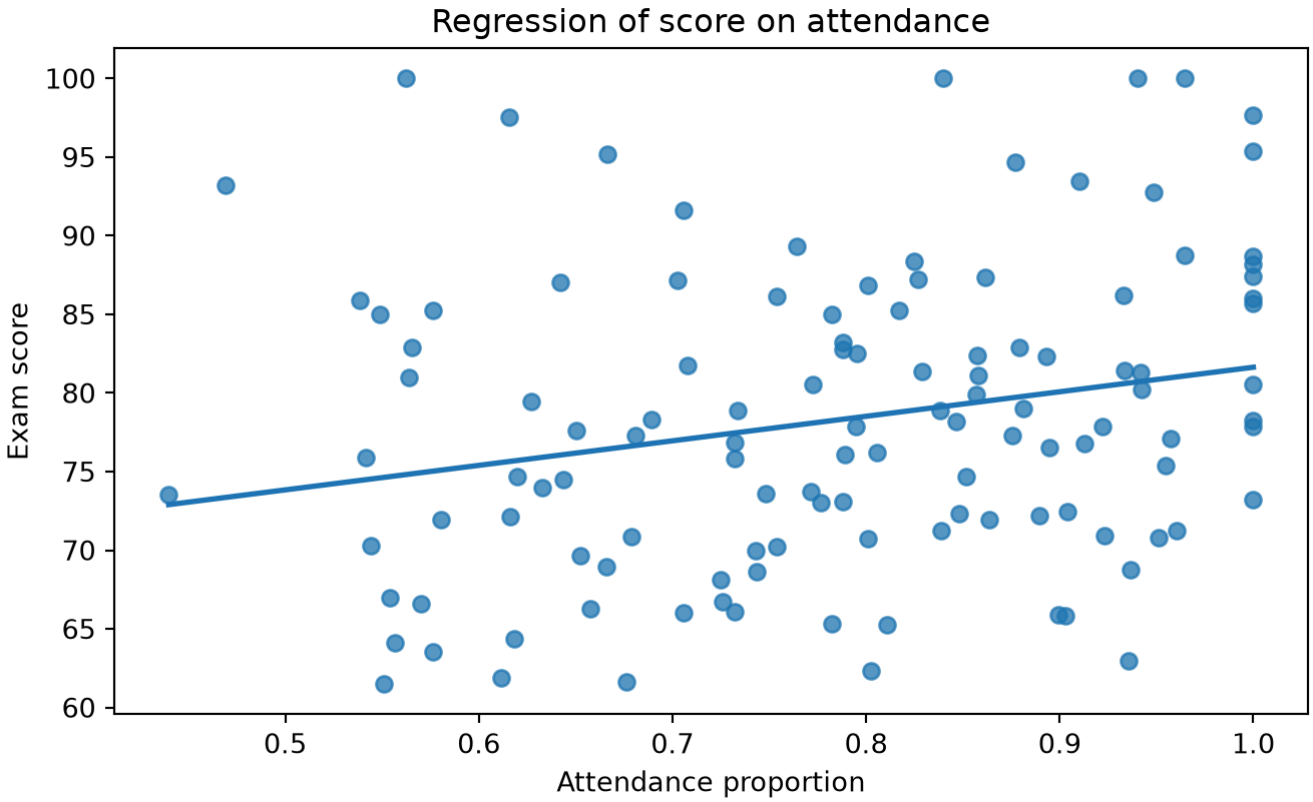


Figure 23:

```
plt.figure(figsize=(8, 4.5))
plt.scatter(attendance_model.fittedvalues, attendance_model.resid, alpha=0.75)
plt.axhline(0, linestyle="--", linewidth=1)
plt.xlabel("Fitted score")
plt.ylabel("Residual")
plt.title("Residuals versus fitted values")
plt.show()
```

Figure 24:

13.6 Solution 6

```
new_study = pd.DataFrame({"study_hours": [1, 5, 10, 20]})
new_study_X = sm.add_constant(new_study, has_constant="add")
preds = q3_model.get_prediction(new_study_X).summary_frame()

pd.concat([new_study, preds], axis=1)
```

	study_hours	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	1	69.747701	1.191850	67.387513	72.107889	54.740690	84.754713
1	5	77.211930	0.694169	75.837286	78.586574	62.328061	92.095799
2	10	86.542215	1.165316	84.234572	88.849858	71.543378	101.541053
3	20	105.202786	3.153003	98.958982	111.446591	89.120964	121.284609

```
students["study_hours"].min(), students["study_hours"].max()
```

```
(np.float64(0.9180250638343775), np.float64(17.38498807122733))
```

Predictions far outside the observed range rely on the assumption that the same linear relationship continues beyond the data. That assumption is often unjustified.

14 Mini reference: choosing a basic method

Data situation	Common method	Example null hypothesis
One numerical variable, compare mean to a value	One-sample t-test	\(\mu=\mu_0\)
One numerical variable, two independent groups	Welch two-sample t-test	\(\mu_1-\mu_0=0\)
One numerical variable, paired before/after observations	Paired t-test	\(\mu_{difference}=0\)
Two numerical variables	Correlation or simple regression	\(\rho=0\) or \(\beta_1=0\)
Numerical outcome and one numerical predictor	Simple linear regression	\(\beta_1=0\)
Binary outcome and predictors	Logistic regression	coefficient equals 0

This table is only a starting point. The right method depends on study design, measurement, assumptions, and the research question.

15 Glossary

Alternative hypothesis: the claim compared against the null **hypothesis**, often representing an effect or difference.

Bias: systematic error in an estimate or study design.

Confidence interval: an interval produced by a method that captures the true parameter at a stated long-run rate when assumptions hold.

Correlation: a unitless measure of linear association between two numerical **variables**.

Estimator: a rule for estimating a population parameter from sample data.

Null hypothesis: a baseline **hypothesis** used to define the reference distribution for a test statistic.

p-value: probability, assuming the null is true, of observing a test statistic as extreme as or more extreme than the observed statistic.

Parameter: a numerical feature of a population or **data-generating process**.

Power: probability that a test rejects the null **hypothesis** when a specified alternative is true.

Regression slope: estimated change in the expected outcome for a one-unit increase in a predictor.

Residual: observed outcome minus fitted outcome.

Sample: observed subset of data.

Sampling distribution: distribution of a statistic over repeated samples.

Standard error: standard deviation of a statistic's sampling distribution.

Statistic: a numerical summary computed from a sample.

Type I error: rejecting a true null **hypothesis**.

Type II error: failing to reject a false null **hypothesis**.

16 Final checklist for students

Before reporting an analysis, ask:

1. What is the research question?
2. What are the observational units?
3. What are the outcome and predictor **variables**?
4. What parameter is being estimated or tested?
5. What assumptions does the method require?
6. What is the estimate?
7. What is the uncertainty: standard error, **confidence interval**, or p-value?
8. Is the effect practically meaningful?
9. Does the study design support a causal interpretation?
10. What limitations should be stated?

17 Suggested class flow

A 75-minute class can use this sequence:

Time	Topic	Activity
10 min	Probability and random variables	Coin-flip simulation
10 min	Sampling variability	Compare repeated sample means
15 min	Confidence intervals	Compute and interpret a mean interval
20 min	Hypothesis/significance testing	One-sample and two-sample t-tests
15 min	Simple regression	Fit and interpret score-on-study-hours model
5 min	Limitations	Statistical significance, practical significance, causality

A longer lab can ask students to complete the exercises and write a short report using the interpretation templates.